PACE: Priced Adaptive Control for Evolving Harnesses

Navid Mehrdad

October 2026

Abstract

Agent harnesses contain per-request decisions that nobody prices: which model tier answers, whether a step is verified, whether an answer is refined, which specialist is delegated to. Recent work evolves the harness at design time or trains the orchestrator with an outcome-only reward; we study the complementary runtime problem. The harness is frozen within a generation, and a small, cost-priced controller is learned at its decision points from logged traffic. Three pieces make this safe to deploy: a policy trained offline under a budget whose price is a learned Lagrange multiplier; an off-policy promotion gate that admits a candidate only when doubly-robust estimates with bootstrap intervals certify its quality and cost; and priced backward- and forward-transfer metrics for the harness changing underneath the controller. On CRMArena-Pro (300 B2B and 150 B2C tasks, every task on three tiers) the learned policy beats every fixed tier held-out on quality and cost at once, the gate's decision depends on the logged draw, with an interval that narrows as $1/\sqrt{n}$ in the number of logged tasks, and two further suites, τ-bench and EvoHarnessBench's hosted service, reproduce the tier-level picture. A continually trained orchestrator over a growing harness has the lowest regret of every learned arm and bounded, mostly positive transfer. Negative results on multi-agent decomposition, process verification, self-refinement and a GRPO-trained LLM controller are reported as results, each with its cause.

1 Introduction

An agent harness is the scaffold around a model: the loop that decides which tool runs next, which model answers, whether a step is checked, when the episode ends. Inside that loop sit decisions made per request by a rule somebody wrote: "always use the large model", "verify every sub-result", "refine if confidence is low". Each decision has a price in dollars and latency, and each is made without reference to that price. Figure 1 shows where these decisions sit and what this paper puts around them.

Two lines of recent work optimise the harness rather than the request. The design-time line evolves the harness itself: the Darwin Gödel Machine [46], GEPA [2], DarwinX [47], Meta-Harness [21], EnvHarness [12] and HarnessX [6] search over designs, and EvoHarnessBench [16] grows a harness $H_t = \{\text{tools}, \text{skills}, \text{specialist agents}\}$ by stages and measures what a fixed agent policy forgets or gains, reporting -34.7% backward transfer from agent-pool expansion alone. The orchestrator-training line trains the policy that assembles the system. MAS-Orchestra [15] generates a complete multi-agent system in one step and optimises the generator with an outcome-only reward; it reports that this holistic formulation outperforms stepwise orchestration, in which an orchestrator chooses one tool or model call at a time [34], and attributes the gap to long-horizon credit assignment under a single terminal reward. Orchestration reward modelling [39] then takes the sub-agents out of the training loop by scoring the plan before it is executed, and MAS-ProVe [40] works at the other end, verifying each agent call or iteration as it happens, and reports that such process verification helps inconsistently and at high variance.

This paper takes the opposite route to the same difficulty. The decisions are per-step and priced, as in the stepwise setting, but credit is assigned per decision rather than per episode: every served decision logs its propensity (the probability with which it was chosen), the objective is quality minus a learned price of cost, and a candidate policy is valued on logged traffic, which it did not generate, by doubly-robust estimators with bootstrap intervals before it is promoted. Holistic generation avoids the credit-assignment problem by removing the steps; the controller here keeps the steps, addresses the problem with the estimators, and treats per-step verification as one more priced action whose value is estimated rather than assumed. ToolOrchestra's efficiency term is, to our knowledge, the closest prior use of a priced orchestration reward; it is an episode-level reward without a per-decision estimator or a promotion gate, and the stepwise result reported for it by Ke et al. [15] is a transplanted baseline, so the comparison of the two formulations under a common price remains open.

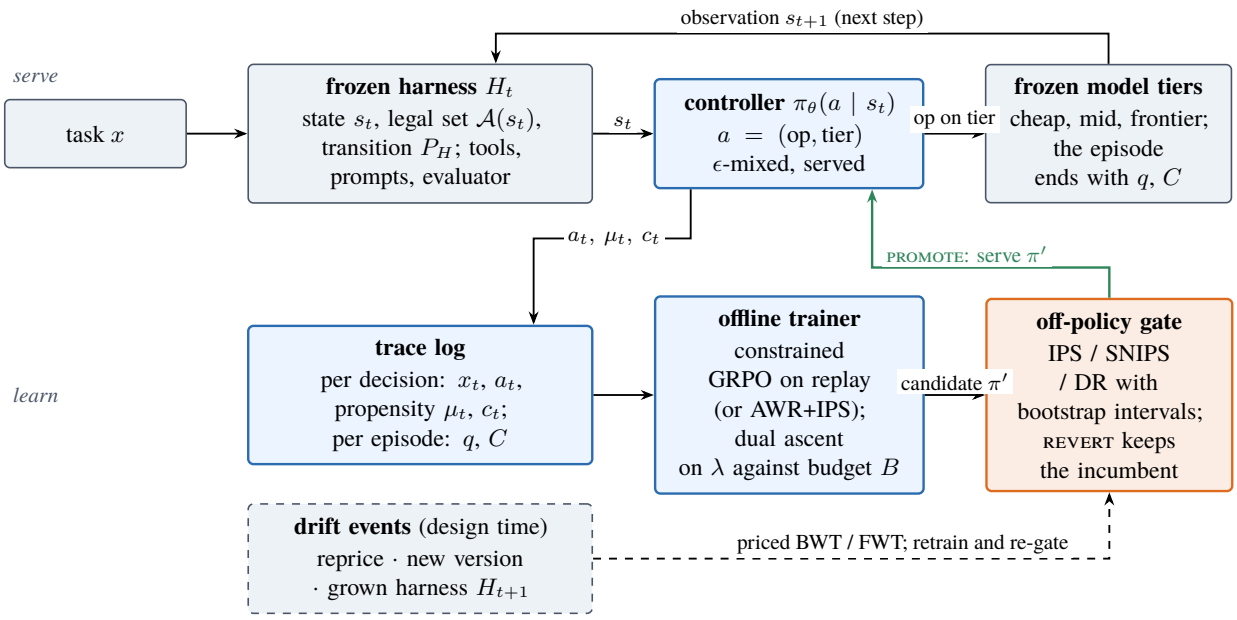


Figure 1: PACE. A frozen harness exposes one decision point per step (which operation, on which model tier). A trace logger records every decision with its propensity and realised cost. A small controller is trained offline from that log under a learned price λ; an off-policy gate scores each candidate on the logged traffic and promotes or reverts it; drift events (a repricing, a new version, a grown harness) are measured by priced transfer. Within a generation nothing in the model weights or the harness structure moves.

Contributions.

- **A priced controller** (Section 3): a policy over (op, tier) pairs whose objective is quality minus a learned price of cost, trained offline by constrained group-relative policy optimisation on replay, the Lagrange multiplier found by damped dual ascent against a per-task budget and the served policy an iterate average.
- **An off-policy promotion gate**: per-decision inverse-propensity (IPS), self-normalised (SNIPS) and doubly-robust (DR) estimators with cross-fitted direct models and percentile-bootstrap intervals, and a two-bar decision rule on the interval edges. We measure its power against sample size on real agent logs rather than assuming it.
- **Priced transfer metrics**: backward and forward transfer in the sense of Lopez-Paz and Ranzato [23], applied to the controller's priced objective as the harness is repriced, re-versioned or grown.
- **Measurements on four suites** (Section 4): 450 CRMArena-Pro tasks on three tiers each, τ-bench retail and airline, three stages of EvoHarnessBench's hosted service, and a 40-day continual-orchestrator study over a frozen, growing harness, together with negative results on decomposition, verification, refinement and LLM controllers, each with its cause stated.

What the measurements say. On CRMArena-Pro the learned per-category policy beats every fixed tier (a cheap, mid or frontier model used for every task) held-out on quality and cost at once (B2B: 0.644 at \$0.079 against the frontier tier's 0.614 at \$0.131), and the same ordering holds on the B2C organisation. The gate's decision on a realised one-arm log depends on the draw, promoting in 30% of redraws at 300 tasks, and its interval narrows as $1/\sqrt{n}$ in the number of logged tasks, as measured from 150 to 300; certifying a half-cent budget margin is a matter of one to two thousand tasks. On τ-bench and EvoHarnessBench the mid tier matches or beats the frontier tier at about half its cost and the per-task oracle (the best tier for each task in hindsight) beats every tier, while a first-step controller has nothing to condition on in τ-bench. The continual orchestrator has the lowest regret of every learned arm, recovers after every harness stage, and has bounded, mostly positive transfer; a fixed mode beats it over the whole stream because of a twelve-day cold start, the days before the gate's first promotion during which the exploration policy is served. The negative results share one cause: in every experiment the limiting factor was the signal available at the decision point, not the learner. Every number in this paper is taken from the committed result files of the public repository.[1]

[1] https://github.com/nvdm59/PACE-Harness

2 Related work

Cost-aware routing. FrugalGPT [5] cascades models by a learned score; RouteLLM [26] learns a router between a strong and a weak model from preference data, and xRouter [29] trains a cost-aware orchestration system with reinforcement learning. These choose a model per query at the first turn and are evaluated by accuracy at a cost. The controller here is a per-step policy with a learned price, served through a gate and evaluated under drift. SkillOrchestra [41] routes to a typed agent in a pool by a learned competence profile with an explicit cost penalty, learned once from traces; it is a design-time baseline in the continual study (Section 4.3).

Harness evolution and orchestrator training. The design-time line [2, 6, 12, 21, 46, 47] and EvoHarnessBench [16] evolve or grow the harness. This paper freezes the harness within a generation, learns inside it, and measures what the controller keeps across generations with the transfer definitions of Lopez-Paz and Ranzato [23]. MAS-Orchestra [15], orchestration reward modelling [39], MAS-ProVe [40] and ToolOrchestra [34] are positioned in Section 1; the multi-agent experiment of Appendix D measures MAS-Orchestra's compute-matched single-agent bar and MAS-ProVe's process-verification finding on one priced log.

Off-policy evaluation and bandits in production. The Decision Service [1] and Horizon [10] established the logging discipline used here: propensities with exploration, and counterfactual evaluation before deployment. The estimators are the per-decision importance sampling of Precup et al. [28], self-normalised importance sampling [35], and the doubly-robust estimators of Dudík et al. [8] and Jiang and Li [14], with the cross-fitting of Chernozhukov et al. [7]. The contribution here is the gate as a deployment rule and the measurement of its power on real agent logs.

Self-evolving agents, test-time compute, and constrained and offline RL. A position paper from the AReaL group [42] names an evolution control plane that chooses among memory, skill, harness and weight updates behind replay-first evaluation; Agent Lightning [24] exposes the proxy seam that makes per-call logging possible; recursive agent optimisation [9] prices delegation effort. Socratic self-refinement [32] and test-time scaling are purchases of compute whose value we measure under a budget (Appendix E). The trainers draw on constrained RL [33, 37], GRPO [31] and DAPO [45], and on offline RL [18, 19, 27]. Appendix K discusses each of these lines at length.

3 Method

3.1 The harness as a frozen, priced decision process

A harness exposes, at each step t of an episode on task x, an observable state s_t and a legal set $\mathcal{A}(s_t)$ of actions $a = (\text{op}, \text{tier})$: an operation the harness can run (for example continue, answer, retrieve, verify, refine, escalate, delegate, re-plan, call a tool or engage a skill; each host exposes its own set of two to six, listed in Appendix M) on a model tier (cheap, mid, frontier). The harness's own transition $s_{t+1} \sim P_H(\cdot \mid s_t, a_t)$ is fixed; the controller $\pi(a \mid s)$ only chooses among what is legal. Each step has a realised cost c_t (tokens times the provider's published per-token price, the list price) and latency ℓ_t, and the episode ends with a quality $q \in [0, 1]$ that is the harness's own score. We write $C = \sum_t c_t$ and $L = \sum_t \ell_t$. The harness is therefore a frozen Markov decision process over the controller's decision points, and the controller is the only thing trained. Figure 2 places this inside the two outer layers that do not move: the model weights, frozen throughout, and the harness structure, frozen within a generation (one version of its tools, prompts, agent pool and prices) and changed only between generations.

Priced objective. A product specification is a budget, so the objective, which we call the priced value, is

$$J(\pi) = \mathbb{E}_\pi[q] - \lambda\, \mathbb{E}_\pi[C] - \mu\, \mathbb{E}_\pi[L], \tag{1}$$

with $\mu = 0$ in every experiment reported here (latency is logged but not priced), and the constrained form

$$\max_\pi \ \mathbb{E}_\pi[q] \quad \text{s.t.} \quad \mathbb{E}_\pi[C] \le B, \qquad \mathcal{L}(\pi, \lambda) = \mathbb{E}_\pi[q] - \lambda\big(\mathbb{E}_\pi[C] - B\big),\ \lambda \ge 0. \tag{2}$$

The price λ is learned by projected dual ascent on an exponential moving average $\hat{c}$ of realised episode cost, with a proportional damping term (the PID-Lagrangian of Stooke et al. [33] without its derivative term; cf. RCPO [37]):

$$\hat{c}_k = \rho\, \hat{c}_{k-1} + (1-\rho)\, \bar{C}_k, \qquad \lambda_k^{\text{int}} = \big[\lambda_{k-1}^{\text{int}} + \eta\,(\hat{c}_k - B)\big]_+, \qquad \lambda_k = \big[\lambda_k^{\text{int}} + k_p\,(\hat{c}_k - B)\big]_+. \tag{3}$$

Primal–dual iterates orbit the budget rather than converging to it, so the served policy is the iterate average $\bar{\pi}(a \mid s) = \frac{1}{|\mathcal{S}|} \sum_{W \in \mathcal{S}} \pi_W(a \mid s)$ over weight snapshots $\mathcal{S}$ from the second half of training [25], and $\bar{\lambda}$ is the mean price over the same half.

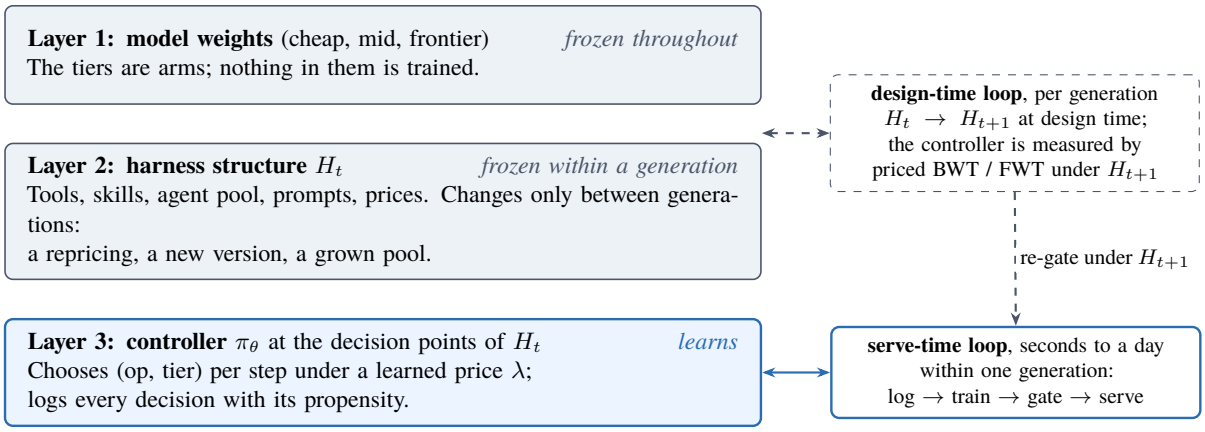


Figure 2: Three layers, two loops. Model weights are frozen. The harness structure H_t (tools, skills, agent pool, prices) is frozen within a generation and changes between generations at design time. The controller is the only layer that learns, in two loops: a serve-time loop (log, train, gate, serve) inside a generation, and a design-time loop across generations, in which priced backward and forward transfer measure what the controller keeps when H_t becomes H_{t+1}.

3.2 Controller and trainer

Policy class and logged propensities. The linear controller is a softmax over soft-clipped logits, mixed with ϵ-exploration:

$$z = L\tanh\big(Wx/L\big), \qquad \pi_\epsilon(a \mid x) = (1-\epsilon)\,\mathrm{softmax}_{\mathcal{A}}(z)_a + \frac{\epsilon}{|\mathcal{A}|}, \tag{4}$$

where $x = \phi(s)$ is a feature vector and the softmax runs over the legal set. The logit clip keeps the softmax steerable by λ and keeps importance ratios bounded. The propensity written to the log is $\mu_t = \pi_\epsilon(a_t \mid x_t)$, exploration included, so every served log is usable for off-policy evaluation, the logging discipline of Bottou et al. [4] and Li et al. [22]. A small language model can take the place of the linear policy (Appendix C); the logging contract is the same.

Constrained GRPO on replay. When the harness can be re-rolled from a logged state (a simulator, or a counterfactual lookup table in which every arm of every task is logged), training samples a logged state s, draws a group of G continuations from the policy *as served* (π_ϵ, so that the cost the dual variable tracks is the served cost), scores each by $R_i = q_i - \lambda C_i$, and uses the group-relative advantage with the group mean as baseline [31]:

$$A_i = R_i - \frac{1}{G}\sum_{j=1}^{G} R_j, \qquad \nabla_W \mathcal{L} = \sum_i A_i \sum_t \nabla_W \log \pi_\epsilon(a_t^i \mid x_t^i), \tag{5}$$

$$\nabla \log \pi_\epsilon(a \mid x) = \frac{(1-\epsilon)\,\pi(a \mid x)}{\pi_\epsilon(a \mid x)}\,\nabla \log \pi(a \mid x). \tag{6}$$

The last factor is the score correction for the ϵ-mixture: the gradient of the mixed policy is the bare softmax's score, scaled by the share of the served probability that the softmax contributes. When no replay exists, the trainer is advantage-weighted regression with inverse-propensity weights at a fixed price (Appendix I); offline RL can only reweight what the log contains.

3.3 The off-policy promotion gate

Estimators. The gate must value a candidate from episodes that another policy generated, which is the off-policy evaluation problem. Three estimators are used: inverse propensity scoring (IPS), which reweights each logged outcome by the ratio of the candidate's probability of the logged action to the logging policy's; its self-normalised form (SNIPS), which divides by the realised sum of the weights instead of by n, trading a small bias for much lower variance; and the doubly-robust estimator (DR), which starts from a direct model of the outcome fitted on the log and applies the importance weights only to that model's residuals, so that it is correct if either the model or the propensities are. For an episode i with per-step rewards r_t^i (the terminal quality, or the per-step cost) and cumulative ratio $\rho_{0:t}^i = \prod_{k\le t} \pi(a_k^i \mid$

$s_k^i)/\mu_k^i$ clipped at $\rho_{\max} = 50$,

$$\hat{V}_{\text{IPS}} = \frac{1}{n}\sum_i \sum_t \rho_{0:t}^i \, r_t^i, \qquad \hat{V}_{\text{SNIPS}} = \sum_t \frac{\sum_i \rho_{0:t}^i \, r_t^i}{\sum_i \rho_{0:t}^i}, \tag{7}$$

$$\hat{V}_{\text{DR}} = \frac{1}{n}\sum_i \Big[\hat{V}(s_0^i) + \sum_t \rho_{0:t}^i \big(r_t^i + \hat{V}(s_{t+1}^i) - \hat{Q}(s_t^i, a_t^i)\big)\Big], \qquad \hat{V}(s) = \sum_a \pi(a \mid s)\, \hat{Q}(s,a), \tag{8}$$

the per-decision importance-sampling estimator of Precup et al. [28], the self-normalised estimator of Swaminathan and Joachims [35], and the sequential doubly-robust (DR) estimator of Jiang and Li [14] (see also 38). The direct model $\hat{Q}$ is a per-feature-bucket mean fitted from the log and iterated under the candidate, which is bucketed fitted-Q evaluation [20]; the clip biases DR only through the direct-model residual. On a first-step contextual bandit (one decision per task, fixed continuation) the bandit form of Dudík et al. [8] applies with a *cross-fitted* direct model:

$$\hat{V}_{\text{DR}} = \frac{1}{n}\sum_i \Big[\sum_a \pi(a \mid x_i)\, \hat{Q}^{(-i)}(x_i, a) + w_i\big(r_i - \hat{Q}^{(-i)}(x_i, a_i)\big)\Big], \qquad w_i = \frac{\pi(a_i \mid x_i)}{\mu_i}, \tag{9}$$

where $\hat{Q}^{(-i)}$ is fitted leaving out task i. The cross-fitting matters at small n [7]: an in-sample direct model shares the logged reward it corrects and shrinks the correction toward zero. For multi-step orchestration logs with two to four decisions over twelve actions, a per-decision variant treats each logged decision as a bandit sample with single-step ratio $w_t = \pi(a_t \mid x_t)/\mu_t$ and reward the episode's quality or cost-to-go; it values changing one decision while the rest of the episode stays as logged, trading bias for ratios bounded by $|\mathcal{A}|/\epsilon$. Every estimate is reported with its effective sample size $\text{ESS} = (\sum_i w_i)^2 / \sum_i w_i^2$ [17]. Appendix P gives the derivations.

Decision rule. Confidence intervals are percentile bootstraps over episodes (over tasks, with all logged arms of a task resampled together, in the bandit case), with 200 resamples and the direct model refitted on every resample so that the DR interval includes direct-model variance. The decision is

$$\textsc{promote} \iff \hat{q}_{\text{DR}}^{\text{lo}} \;\geq\; q_{\text{prev}} - \epsilon_q \quad \text{and} \quad \hat{c}_{\text{DR}}^{\text{hi}} \;\leq\; B + \text{tol}, \tag{10}$$

with q_{prev} the incumbent's quality: its on-policy log mean if it is the behaviour policy, its own DR estimate on the log otherwise, or the DR quality recorded at the incumbent's own promotion after a repricing, since an over-budget incumbent would otherwise set a bar that no budget-respecting candidate can pass. Defaults are $\epsilon_q = 0.03$ and $\text{tol} = \$0.005$ (0.02 in simulator units). The gate uses the interval edges, not the point estimates: a candidate must be certifiably no worse and certifiably within budget.

3.4 Priced transfer under harness drift

Let $R_{i,j}$ be the held-out priced value J_j of the policy in service at the end of stage i, evaluated on stage j, with J_j priced at the stage's own $\bar{\lambda}_j$. Following Lopez-Paz and Ranzato [23],

$$\text{BWT}_j = R_{T,j} - R_{j,j} \;(j < T), \qquad \text{FWT}_j = R_{j-1,j} - b_j, \tag{11}$$

with b_j the value of a uniform policy on stage j. For the two-version case, priced-BWT $= J(\text{frozen on } v_2) - J(\text{frozen on } v_1)$ and recovery $= J(\text{served on } v_2 \text{ after retrain and gate}) - J(\text{frozen on } v_2)$. The distinction from EvoHarnessBench's metrics is the object measured: there, a fixed agent policy under a changing harness; here, the priced value of a controller that keeps learning while the structure around it changes.

3.5 Platform

A host harness implements `reset`, `legal_actions`, `step` and `quality`; a simulable one also implements `snapshot` and `restore`, which is what constrained GRPO on replay needs. Its operations are the ops, its model pool the tiers, its version string a feature. The trace schema has one record per controller decision (features, action, propensity with exploration included, the legal set the behaviour chose from, realised cost and latency, and a state snapshot when the harness is simulable) and one record per episode (terminal quality, the agent's answer, the queries, the evaluator's calls). Every model call is logged with its model, token counts, priced cost, task and purpose, and a hard budget check on the call's worst-case cost runs before the call is sent, so every paid collection is fail-closed and resumable per (task, arm). Four hosts are used: a hand-written simulator with twelve actions over a latent task difficulty (Appendix A);

held-out, 5-fold ($\lambda = 2$ per dollar)	B2B, 300 tasks			B2C, 150 tasks		
	quality	cost	J [fold SD]	quality	cost	J [fold SD]
fixed cheap	0.434	\$0.057	0.320 [0.063]	0.469	\$0.056	0.358 [0.078]
fixed mid	0.482	\$0.126	0.230 [0.026]	0.508	\$0.126	0.255 [0.109]
fixed frontier	0.614	\$0.131	0.353 [0.046]	0.638	\$0.143	0.352 [0.116]
AWR+IPS linear, greedy	0.596	\$0.080	0.437 [0.028]	0.598	\$0.081	0.435 [0.057]
per-category rule (train-fold argmax)	**0.644**	**\$0.079**	**0.486 [0.035]**	**0.636**	**\$0.081**	**0.474 [0.067]**
constrained, $B = \$0.10$, greedy of the mix†	0.644	\$0.103	0.521 [0.073]	0.677	\$0.105	0.549 [0.100]
per-task oracle	0.708	\$0.058	0.592 [0.032]	0.727	\$0.061	0.606 [0.068]

Table 1: CRMArena-Pro, held-out tier policies (5-fold CV over task ids, exact lookups). In both organisations the learned per-category policy beats every fixed arm on quality and cost at once, by three fold SDs on B2B. $^{\dagger}J$ at each fold's learned $\bar{\lambda}$ (mean 1.22 per dollar on B2B, 1.35 on B2C); the B2C constrained policy sits \$0.0002 over the \$0.105 tolerance.

CRMArena-Pro [13], whose ReAct agent [43] is re-expressed as a steppable loop over the benchmark's own tasks, prompts, live CRM connector and evaluator, with actions `CONTINUE@tier` and `ANSWER@tier` over three tiers and 43 features (Appendix B); τ-bench [44]; and the hosted service of EvoHarnessBench [16]. Every setting of every run is in Appendix M.

4 Experiments

Seven experiments exercise the method in increasing realism. The body reports the four that carry the paper's claims and summarises the rest; every experiment is reported in full, with setup, results and limitations, in the appendices. All real-harness collections follow one *counterfactual protocol*: every task is run on every tier with propensity $1/3$ at the first step and a fixed continuation on the chosen tier, so that the held-out quality and cost of any first-step policy is an exact lookup and the off-policy estimators can be checked against the truth. Costs are token counts times list prices, with one exception stated where it occurs. Two reference quantities recur. The *per-task oracle* picks, for each task, the tier with the best logged quality, cheapest among ties; it is an upper bound on any first-step policy and is attainable only with knowledge of the outcome, so the gap between it and the best learned policy measures what the observables at the decision point fail to predict. In the continual study the *per-stage oracle* is a controller trained on fresh exploration episodes of that stage with no gate, a reference rather than a proven optimum, and *regret* is the sum over days of the oracle's held-out priced value minus the serving policy's on the day's stage.

4.1 CRMArena-Pro at 300 B2B and 150 B2C tasks

CRMArena-Pro [13] poses CRM tasks over a live Salesforce organisation with a program-checked answer, in two sandbox organisations, B2B and B2C, with different schemas and task mixes; the harness is its own ReAct agent, re-expressed as a steppable loop, and the learned decision is the first-step tier from 43 step-0 features, chiefly the task category. The first collection, 50 tasks, is where the controller memorised: no learned policy clearly beat the best fixed tier held-out, and the gate said so (Appendix B). The collection was then resumed on the same seeded, category-stratified sample to 150 and 300 B2B tasks (900 episodes), and a second collection of 150 tasks was run on the B2C organisation with the same protocol. Everything was re-run with the same code and folds. Policies are trained and evaluated by 5-fold cross-validation over task ids, stratified by category: a held-out number is the mean over the five test folds, and the bracketed fold SD is the standard deviation of J across them. Four policies are compared with the fixed tiers and the oracle: the *per-category rule*, which assigns to each category the tier with the best training-fold J; the *AWR+IPS linear* controller of Appendix I, served greedily; and the *constrained* controller of Section 3.2, trained under the budget B on the lookup table and served as the greedy action of its iterate-averaged policy.

The held-out ordering is stable across scale and organisation (Table 1). At 50 tasks no learned controller clearly beat the best fixed arm; at 150 the per-category rule led by a little over one fold SD; at 300 its J of 0.486 is 0.13 above the best fixed arm (frontier, 0.353) at three fold SDs, and the same rule leads on B2C, a different data schema and task mix, by 0.12 over its best fixed arm. The constrained controller reaches the frontier tier's quality at 22% less cost on B2B and exceeds it on B2C. **The tier means keep moving as tasks are added** (B2B mid $0.533 \to 0.482$ from 150 to 300 tasks; frontier $0.616 \to 0.614$), which is why the held-out comparison, not the tier table, is the result. **Repricing drift replicates on both organisations.** Held-out, a frontier price rise of $1.5\times$ costs the frozen policy -0.050 in priced value on B2B and -0.049 on B2C, of which a retrain recovers $+0.035$ and $+0.027$; halving the mid price is neutral ($+0.002$ on both) and the retrain gains $+0.031$ and $+0.030$ by moving traffic to mid; tripling the cheap price

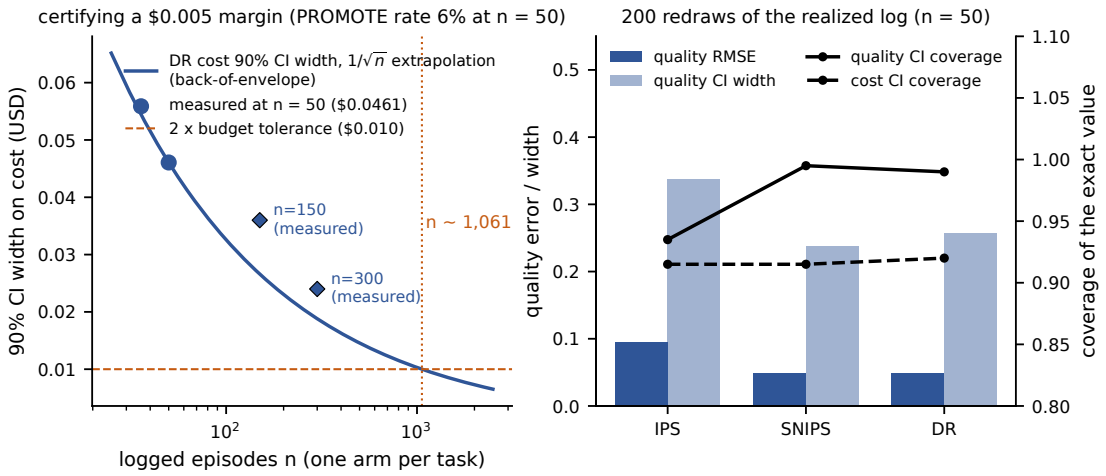


Figure 3: Gate power. Left: the DR cost interval width measured at $n = 50$ with its $1/\sqrt{n}$ extrapolation against twice the budget tolerance, and the widths later measured at $n = 150$ and 300 (diamonds). The two measurements sit about 30% above the curve anchored at 50 and on the curve anchored at 150; the vertical marker is the 50-anchored estimate of the tasks needed, and the 300-anchored one is about 1,700. Right: estimator RMSE, interval width and coverage over 200 redraws of the realised log at $n = 50$.

costs -0.038 and -0.047, and the retrained policy on B2B recovers $+0.044$ but leaves the budget. Every retrained candidate is REVERTed by the gate on the repriced realised log, on the quality bar recorded at the incumbent's promotion, the cost bound, or both: at this sample size the gate is the conservative element of the loop, which is its design.

4.2 The gate's sample-size result

The gate is run on the *realised log*, one uniformly drawn tier per task, which is what a deployment log looks like, and then over 200 independent redraws of that log (a fresh uniform draw of one tier per task each time), so that the decision's dependence on the draw is measured rather than assumed. **The gate's answer depends on the draw, and more data moves the rate, not the decision.** On the seed-0 realised log at 300 tasks the DR estimate for the all-task constrained policy is 0.607 [0.525, 0.668] in quality and \$0.095 [0.087, 0.113] in cost, and the gate REVERTS on the cost bound; the policy's exact cost is \$0.098 against a \$0.100 budget, so the candidate sits inside the budget by less than the tolerance and the interval cannot certify it. Over 200 redraws the promotion rate is 30% at 300 tasks (10% at 150, 6% at 50) and 8% on B2C; the DR cost interval has mean width 0.024 (0.030 on B2C) with 96% coverage of the exact value, and the DR quality estimate has RMSE 0.029 (Figure 3). From 150 to 300 tasks the width fell from 0.036 to 0.024, which is what the $1/\sqrt{n}$ law predicts from the 150-task point (0.025); the projection made from the 50-task width had been optimistic (0.019 at 300), and the promotion rate exceeded the ten to fifteen percent projected from it. Certifying a half-cent margin needs an interval near \$0.01, which from the 300-task width is about 1,700 tasks. That number is a result rather than a limitation: it says what a production gate on this kind of log costs in traffic, and it is why Section 5 recommends certifying cost and treating quality as a floor.

4.3 The continual orchestrator

The centrepiece asks EvoHarnessBench's question turned around: when the harness $H_t = \{\text{tools}, \text{skills}, \text{agents}\}$ grows under an orchestrator that keeps learning, what happens to the policy's priced backward and forward transfer, and does the gate keep it safe without making it too slow to matter? The multi-agent structure of Appendix D is expressed as a typed harness (18 actions over an agent pool, tools and skills; 88 features) on a simulator calibrated to the real logs (Appendix Q) and run for 40 days of 96 tasks under four pre-registered stages: a repricing, new specialists including a better verifier, new tools, and skills with a second planner. The continual arm serves its incumbent with $\epsilon = 0.1$, trains daily by constrained GRPO on the last five days of replay, and is gated daily by sequential DR against the incumbent. Comparators on the same traffic: the frozen day-5 policy, a SkillOrchestra-style router learned once and one re-learned per stage, the fixed modes `always_verify` and `never_verify`, and the per-stage oracle defined above. Appendix F gives the stages, the day loop, every table, a mild-schedule sensitivity run and the verifier ablation.

The online learner has the lowest regret of every learned arm and recovers after every stage (Figure 4): cumulative regret 1.26, against 2.60 for the frozen policy and 3.03–3.46 for the static routers, ending each stage within

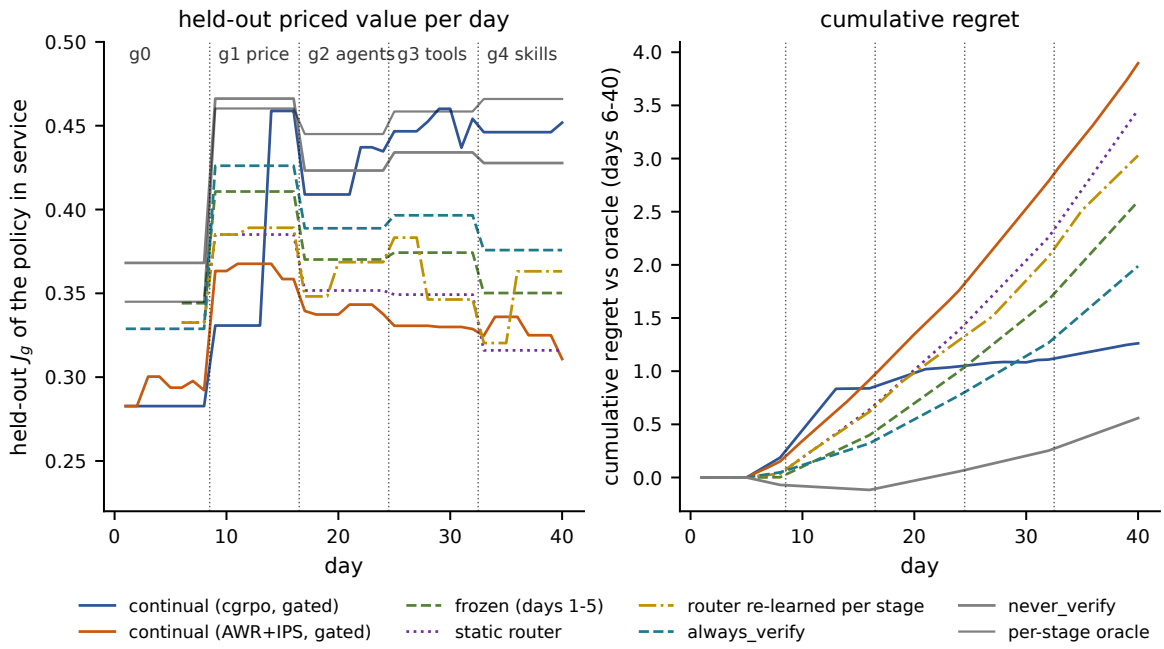


Figure 4: The continual orchestrator over 40 days (default schedule). Left: held-out J_g of the policy in service each day; the grey upper line is the per-stage oracle. Right: cumulative regret to the oracle from day 6. The continual arm is the only learned arm that recovers after every stage; the fixed mode `never_verify` has the lowest regret of all.

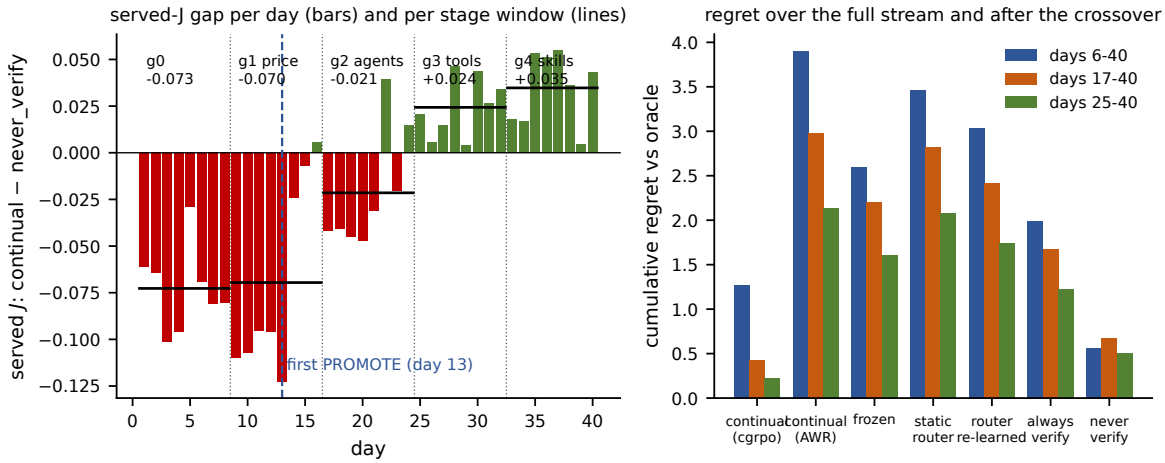


Figure 5: The crossover. Left: served J of the continual arm minus `never_verify`, per day (bars) and averaged per stage window (black lines), with the harness events (dotted) and the first promotion (dashed, day 13). Right: cumulative held-out regret to the oracle over the full stream (days 6–40) and restricted to after the agent-pool event (17–40) and after the tools event (25–40).

0.014 of the oracle. **A fixed mode beats it over the whole stream, and that headline is a cold-start artefact.** `never_verify` has regret 0.56 (0.74 ± 0.08 against the learner's 1.06 ± 0.21 over 5 seeds): the learner spends twelve days on the uniform exploration policy before its first promotion, crosses the fixed mode around day 22–25 (Figure 5), and from day 25 leads it (0.22 against 0.50; $+0.031 \pm 0.007$ at the last stage over 5 seeds), while the fixed mode falls further below the oracle at each capability stage because it cannot use what arrives. **The gate is the conservative element.** 32 of 40 candidates were reverted, 9 rightly and 23 as false alarms; at $n \approx 500$ episodes the DR quality interval is ± 0.04 or wider, while the cost interval is \$0.008 wide and its criterion is the one that binds. **Orchestrator-level BWT is bounded and mostly positive**, because the structure is frozen and only the policy moves; FWT is a property of the policy in service, not of anticipation. **Verification helps only when the verifier is informative and the retry**

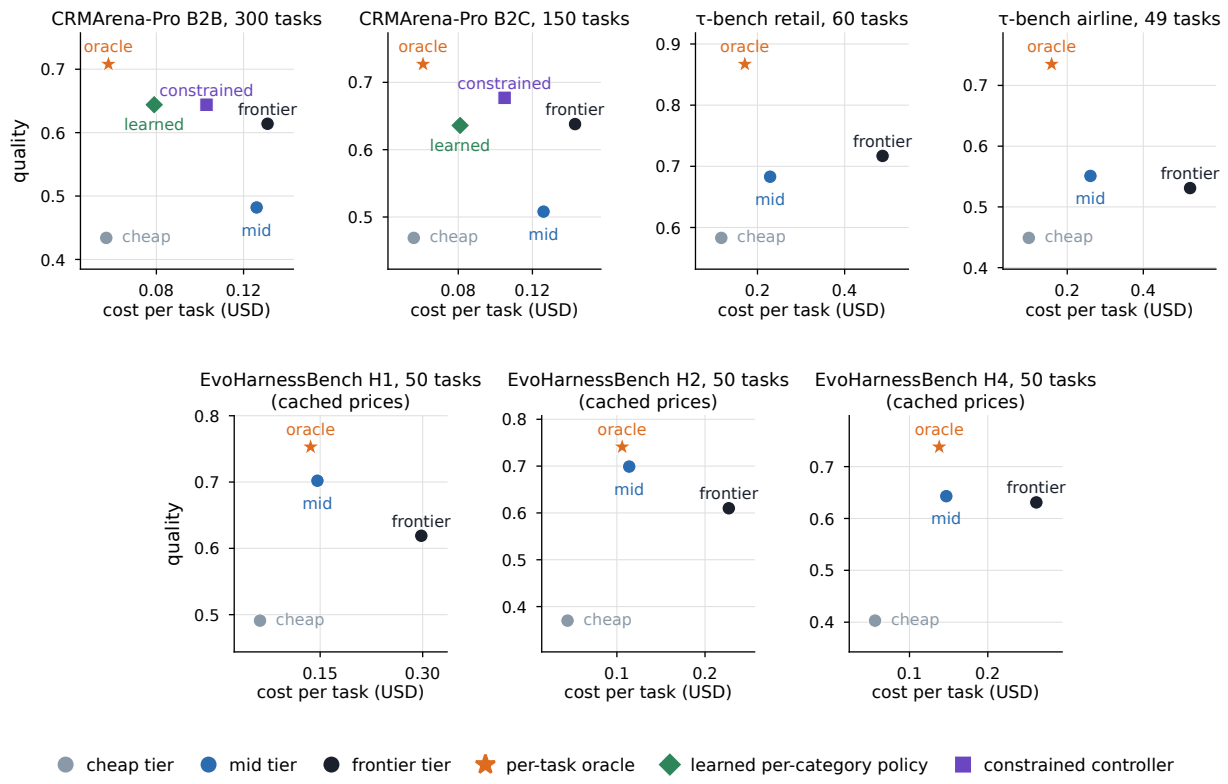


Figure 6: Quality against cost per task on every suite. Fixed tiers (circles), the per-task oracle (star) and, on CRMArena-Pro, the learned per-category policy (diamond) and the budgeted controller (square). On every suite the oracle sits above and to the left of every tier; on τ-bench airline and all three EvoHarnessBench stages the mid tier beats the frontier tier outright at about half its cost. EvoHarnessBench costs are at cached prices.

does not come out of the plan's call budget (Appendix F.2): under the committed configuration `always_verify` is below `never_verify` by 0.033 ± 0.001 in quality; with an informative verifier (one whose flags carry information about correctness, which the measured verifier's do not) and the per-episode cap on role calls raised from 8 to 12, it is above by 0.029 ± 0.001, for $+18\%$ cost. Three follow-ups (Appendix H) add that RAO-style delegation credit buys over-delegation (-0.02 to -0.04 in stationary J at every stage), that a meta-learned initialisation halves the learner's regret ($1.06 \pm 0.21 \rightarrow 0.47 \pm 0.08$) without changing its first promotion day, and that a skill handbook for the cheap tier does nothing measurable once a no-notes control is run.

4.4 Two further suites: τ-bench and EvoHarnessBench

To see whether the tier-level findings depend on the shape of CRMArena-Pro, the same protocol was run on τ-bench [44], a multi-turn conversation with a simulated customer under a policy document (retail, 60 tasks; airline, 49), and on EvoHarnessBench's hosted service [16], which grades the final state of a live enterprise environment with hidden verifiers and grows the tool set by stages (50 tasks at each of stages 1, 2 and 4, under each stage's own allowlist, with prompt caching on the fixed prefix, so its cost column is at cached prices). Figure 6 places every suite on one quality–cost plane; the full tables are in Appendix G.

The per-task headroom is largest where episodes are longest. On τ-bench retail the oracle reaches 0.867 at \$0.171 against the frontier tier's 0.717 at \$0.487, because the tiers fail on different tasks (52 of 60 solved by at least one tier, 26 by all three); on airline the mid tier beats the frontier tier outright (0.551 against 0.531 at half the cost). On EvoHarnessBench the mid tier beats the frontier tier on verifier score at every stage (0.702 vs 0.619, 0.699 vs 0.610, 0.643 vs 0.631) and the oracle beats every tier at mid-tier cost, choosing the frontier tier for only 4 tasks in 50; the ordering is the same at the three harness sizes. **None of the τ-bench headroom is reachable by a first-step controller, and the log says why.** The task is revealed by the simulated user over the conversation, so at step 0 the feature vector is the domain one-hot and every learned first-step policy collapses to a fixed arm; the realised-log gate reverts and

5% of redraws promote. The decision that would capture the headroom is a mid-episode tier switch conditioned on the conversation so far, which this protocol does not log. No controller was trained on the EvoHarnessBench logs, and every task ran under its own stage's harness, so that collection feeds the benchmark's forward-transfer group only (tasks scored under the stage that introduced them).

4.5 Negative results, in brief

Three further experiments returned negative results. Each is reported in full in the appendices with its cause; the causes are what make them results.

A GRPO-trained LLM controller did not find the budgeted optimum (Appendix C). A 1.7B-parameter model with a LoRA adapter, trained by GRPO on the simulator under the same dual-ascent price, produced near-identical rollout groups (83–97% of groups degenerate, meaning every completion in the group was the same, which makes the group-relative advantage zero), so the only learning signal was λ moving the mean reward, and the policy jumped between arms instead of conditioning on the state. An SFT warm start on oracle labels did the conditioning (100% held-out agreement), and GRPO's contribution was the price, $\lambda \approx 0.44$. Platform controls settled the cause: with the price held fixed, TRL, verl on Ray and NeMo-RL all learn the split and agree within 0.001 in priced value, and DAPO's dynamic sampling sharpens it; the moving price, not the sampler or the stack, broke the run.

A planner/worker/verifier decomposition scores below a single agent (Appendix D): 0.480 against 0.608 on the same 50 tasks, dominated on the priced frontier; this is MAS-Orchestra's bar, a single agent given the same compute, and it is not cleared. The verifier is an alarm, not a judge, flagging 92% of wrong and 96% of correct episodes; an orchestration reward model on process features has an out-of-fold AUROC (area under the ROC curve, 0.5 being chance) of 0.469 against 0.821 for the task category alone.

Priced self-refinement is dominated by buying a better tier up front (Appendix E): the cheap probe that decomposes an answer into claims with self-reported confidences barely predicts a wrong answer (AUROC 0.65), a cheap refinement round breaks as many answers as it fixes, and the mid tier at the first turn scores 0.66 at \$0.115 where the best cheap-refinement policy at that price scores 0.49.

5 Discussion

The signal is the bottleneck. The same pattern appears in every experiment. On CRMArena-Pro the per-task oracle is far above every arm, but the only observable the controller has at step 0 is the category; on τ-bench it is the domain alone. The orchestration reward model finds no out-of-fold signal in process features while the category has AUROC 0.82; the verifier flags nearly everything; the self-refinement probe barely sees whether the cheap answer is wrong; the LLM controller's GRPO stage had no advantage signal because its rollout groups were identical. In the continual study the gate's quality criterion has too little signal at $n \approx 500$ to tell an acceptable candidate from the incumbent, while the cost criterion, whose signal is a sum of logged dollars, works. The learners are not the limiting factor; the observables at the decision point are. Three practical consequences follow. Log richer state at the decision point, which for τ-bench means the conversation so far and for the CRM tasks means the first query's result. Certify cost and treat quality as a floor against regression rather than as an improvement test, because the cost interval closes at a few hundred episodes and the quality interval does not. Keep candidates close to the behaviour policy, since the estimator's bias, not its variance, is what promoted the one bad arm.

When does a learned orchestrator earn its keep? Only when a harness change moves the right fixed policy. A repricing moves a fixed policy's cost but not its ranking; a new specialist or tool moves the ranking, and only a policy that can see the new capability can use it. If the harness is stable and someone knows the right fixed mode, the learner's whole-stream regret is its cold start, its recovery days and its gate delays, and the fixed mode wins; once the pool has grown past what the fixed mode can use, the learner wins while paying an exploration tax the fixed mode does not. Which fixed mode is right depends on the harness, and that is the argument for learning it: with the measured verifier the right mode is never to verify, with an informative one and a looser cap it is always to verify, and the budgeted learner lands between them under every variant. The meta-learned initialisation of Appendix H halves the cold start on seen stages and does nothing on the unseen one, which is the shape one would expect from a design-time loop that learns from past generations; Appendix J states what a real-traffic continuation would need.

Limitations. The real-harness evidence is 450 CRMArena-Pro tasks, 109 τ-bench tasks and 150 EvoHarnessBench tasks, each on three tiers, from one seed of each collection; held-out differences between policies are of the order of the

across-fold SDs below 300 tasks. The continual study is a simulator calibrated to 130 real multi-agent episodes, with its stage effect sizes assumed; its all-arms run is one seed per schedule and only the ablation and the follow-ups have five. Each GPU run ran once. The same tier on the same tasks moved by 0.12 between two runs a day apart, which bounds what any held-out comparison at this scale can certify. Appendix L lists every threat to validity we identified, and Appendix M every setting of every run.

Acknowledgements

The authors of CRMArena-Pro released the benchmark, its prompts, evaluator and read-only sandbox that made the real-harness experiments possible; the authors of EvoHarnessBench made their hosted service available for the tools-axis collection.

References

[1] Alekh Agarwal, Sarah Bird, Markus Cozowicz, Luong Hoang, John Langford, Stephen Lee, Jiaji Li, Dan Melamed, Gal Oshri, Oswaldo Ribas, Siddhartha Sen, and Alex Slivkins. Making contextual decisions with low technical debt. *arXiv preprint arXiv:1606.03966*, 2016.

[2] Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.

[3] Yavuz Bakman, Duygu Nur Yaldiz, Sungmin Kang, Tuo Zhang, Baturalp Buyukates, Salman Avestimehr, and Sai Praneeth Karimireddy. Reconsidering LLM Uncertainty Estimation Methods in the Wild. *arXiv preprint arXiv:2506.01114*, 2025.

[4] Léon Bottou, Jonas Peters, Joaquin Quiñonero-Candela, Denis X. Charles, D. Max Chickering, Elon Portugaly, Dipankar Ray, Patrice Simard, and Ed Snelson. Counterfactual reasoning and learning systems: The example of computational advertising. *Journal of Machine Learning Research*, 14:3207–3260, 2013. arXiv:1209.2355.

[5] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

[6] Tingyang Chen, Shuo Lu, Kang Zhao, Weicheng Meng, Hanlin Teng, Tianhao Li, Chao Li, Xule Liu, Jian Liang, Zhizhong Zhang, Yuan Xie, Heng Qu, Kun Shao, and Jian Luan. HarnessX: A Composable, Adaptive, and Evolvable Agent Harness Foundry. *arXiv preprint arXiv:2606.14249*, 2026.

[7] Victor Chernozhukov, Denis Chetverikov, Mert Demirer, Esther Duflo, Christian Hansen, Whitney Newey, and James Robins. Double/debiased machine learning for treatment and structural parameters. *The Econometrics Journal*, 21(1):C1–C68, 2018. arXiv:1608.00060.

[8] Miroslav Dudík, John Langford, and Lihong Li. Doubly robust policy evaluation and learning. In *Proceedings of the 28th International Conference on Machine Learning (ICML)*, 2011. arXiv:1103.4601.

[9] Apurva Gandhi, Satyaki Chakraborty, Xiangjun Wang, Aviral Kumar, and Graham Neubig. Recursive Agent Optimization. *arXiv preprint arXiv:2605.06639*, 2026.

[10] Jason Gauci, Edoardo Conti, Yitao Liang, Kittipat Virochsiri, Yuchen He, Zachary Kaden, Vivek Narayanan, Xiaohui Ye, Zhengxing Chen, and Scott Fujimoto. Horizon: An open source applied reinforcement learning platform. *arXiv preprint arXiv:1811.00260*, 2018.

[11] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2106.09685.

[12] Chengsong Huang, Zifeng Wang, Rujun Han, Jun Yan, Yanfei Chen, Zoey CuiZhu, Ke Jiang, Peng Xia, Han Yu, Yufan Zhuang, Yifei Ming, Jiaqi Pan, Bhavana Dalvi Mishra, Jiaxin Huang, Burak Gokturk, Tomas Pfister, and Chen-Yu Lee. EnvHarness: Awakening Static Worlds for Agent Learning. *arXiv preprint arXiv:2608.19880*, 2026.

[13] Kung-Hsiang Huang, Akshara Prabhakar, Onkar Thorat, Divyansh Agarwal, Prafulla Kumar Choubey, Yixin Mao, Silvio Savarese, Caiming Xiong, and Chien-Sheng Wu. CRMArena-Pro: Holistic assessment of LLM agents across diverse business scenarios and interactions. *arXiv preprint arXiv:2505.18878*, 2025.

[14] Nan Jiang and Lihong Li. Doubly robust off-policy value evaluation for reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 2016. arXiv:1511.03722.

[15] Zixuan Ke, Yifei Ming, Austin Xu, Ryan Chin, Xuan-Phi Nguyen, Prathyusha Jwalapuram, Jiayu Wang, Semih Yavuz, Caiming Xiong, and Shafiq Joty. MAS-Orchestra: Understanding and Improving Multi-Agent Reasoning Through Holistic Orchestration and Controlled Benchmarks. *arXiv preprint arXiv:2601.14652*, 2026.

[16] Zixuan Ke, Vaidehi Patil, Haizhou Shi, Yang Li, Ye Liu, Sarath Shekkizhar, Anurag Koul, Jiayu Wang, Xuan Phi Nguyen, Semih Yavuz, Mohit Bansal, and Shafiq Joty. EvoHarnessBench: Can Your Agents Keep Pace with an Evolving Harness? *arXiv preprint arXiv:2609.04280*, 2026.

[17] Leslie Kish. *Survey Sampling*. John Wiley & Sons, 1965.

[18] Ilya Kostrikov, Ashvin Nair, and Sergey Levine. Offline reinforcement learning with implicit Q-learning. *arXiv preprint arXiv:2110.06169*, 2021.

[19] Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative Q-learning for offline reinforcement learning. In *Advances in Neural Information Processing Systems 33*, 2020. arXiv:2006.04779.

[20] Hoang M. Le, Cameron Voloshin, and Yisong Yue. Batch policy learning under constraints. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019. arXiv:1903.08738.

[21] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-Harness: End-to-End Optimization of Model Harnesses. *arXiv preprint arXiv:2603.28052*, 2026.

[22] Lihong Li, Wei Chu, John Langford, and Xuanhui Wang. Unbiased offline evaluation of contextual-bandit-based news article recommendation algorithms. In *Proceedings of the 4th ACM International Conference on Web Search and Data Mining (WSDM)*, 2011. arXiv:1003.5956.

[23] David Lopez-Paz and Marc'Aurelio Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems 30*, 2017. arXiv:1706.08840.

[24] Xufang Luo, Yuge Zhang, Zhiyuan He, Zilong Wang, Siyun Zhao, Dongsheng Li, Luna K. Qiu, and Yuqing Yang. Agent Lightning: Train ANY AI Agents with Reinforcement Learning. *arXiv preprint arXiv:2508.03680*, 2025.

[25] Angelia Nedić and Asuman Ozdaglar. Subgradient methods for saddle-point problems. *Journal of Optimization Theory and Applications*, 142(1):205–228, 2009.

[26] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024.

[27] Xue Bin Peng, Aviral Kumar, Grace Zhang, and Sergey Levine. Advantage-weighted regression: Simple and scalable off-policy reinforcement learning. *arXiv preprint arXiv:1910.00177*, 2019.

[28] Doina Precup, Richard S. Sutton, and Satinder Singh. Eligibility traces for off-policy policy evaluation. In *Proceedings of the 17th International Conference on Machine Learning (ICML)*, 2000.

[29] Cheng Qian, Zuxin Liu, Shirley Kokane, Akshara Prabhakar, Jielin Qiu, Haolin Chen, Zhiwei Liu, Heng Ji, Weiran Yao, Shelby Heinecke, Silvio Savarese, Caiming Xiong, and Huan Wang. xRouter: Training Cost-Aware LLMs Orchestration System via Reinforcement Learning. *arXiv preprint arXiv:2510.08439*, 2025.

[30] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[31] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[32] Haizhou Shi, Ye Liu, Bo Pang, Zeyu Leo Liu, Hao Wang, Silvio Savarese, Caiming Xiong, Yingbo Zhou, and Semih Yavuz. SSR: Socratic Self-Refine for Large Language Model Reasoning. *arXiv preprint arXiv:2511.10621*, 2025.

[33] Adam Stooke, Joshua Achiam, and Pieter Abbeel. Responsive safety in reinforcement learning by PID lagrangian methods. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020. arXiv:2007.03964.

[34] Hongjin Su, Shizhe Diao, Ximing Lu, Mingjie Liu, Jiacheng Xu, Xin Dong, Yonggan Fu, Peter Belcak, Hanrong Ye, Hongxu Yin, Yi Dong, Evelina Bakhturina, Tao Yu, Yejin Choi, Jan Kautz, and Pavlo Molchanov. ToolOrchestra: Elevating Intelligence via Efficient Model and Tool Orchestration. *arXiv preprint arXiv:2511.21689*, 2025.

[35] Adith Swaminathan and Thorsten Joachims. The self-normalized estimator for counterfactual learning. In *Advances in Neural Information Processing Systems 28*, 2015.

[36] Arnuv Tandon, Karan Dalal, Xinhao Li, Daniel Koceja, Marcel Rød, Sam Buchanan, Xiaolong Wang, Jure Leskovec, Sanmi Koyejo, Tatsunori Hashimoto, Carlos Guestrin, Jed McCaleb, Yejin Choi, and Yu Sun. End-to-End Test-Time Training for Long Context. *arXiv preprint arXiv:2512.23675*, 2025.

[37] Chen Tessler, Daniel J. Mankowitz, and Shie Mannor. Reward constrained policy optimization. *arXiv preprint arXiv:1805.11074*, 2018.

[38] Philip S. Thomas and Emma Brunskill. Data-efficient off-policy policy evaluation for reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning (ICML)*, 2016. arXiv:1604.00923.

[39] King Yeung Tsang, Zihao Zhao, Vishal Venkataramani, Haizhou Shi, Zixuan Ke, Semih Yavuz, Shafiq Joty, and Hao Wang. Reward Modeling for Multi-Agent Orchestration. *arXiv preprint arXiv:2606.13598*, 2026.

[40] Vishal Venkataramani, Haizhou Shi, Zixuan Ke, Austin Xu, Xiaoxiao He, Yingbo Zhou, Semih Yavuz, Hao Wang, and Shafiq Joty. MAS-ProVe: Understanding the Process Verification of Multi-Agent Systems. *arXiv preprint arXiv:2602.03053*, 2026.

[41] Jiayu Wang, Yifei Ming, Zixuan Ke, Shafiq Joty, Aws Albarghouthi, and Frederic Sala. SkillOrchestra: Learning to Route Agents via Skill Transfer. *arXiv preprint arXiv:2602.19672*, 2026.

[42] Ran Yan, Wei Fu, Jiale Li, Shusheng Xu, Zhiyu Mei, Jiaxuan Gao, Jiarui Zhang, Wentai Zhang, Hao Dai, Xujie Shen, Chuyi He, Zhen Pu, Jun Mei, Zhiyao Lin, Haitao Wang, Zhiqiang Ding, Jiawei Zhang, Huaijie Wang, Ruida Xu, Honghua Dong, Youhe Jiang, Yi Wu, Tongkai Yang, and Binhang Yuan. Next-Generation Agentic Reinforcement Learning Systems Enable Self-Evolving Agents. *arXiv preprint arXiv:2607.01120*, 2026.

[43] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.

[44] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv preprint arXiv:2406.12045*, 2024.

[45] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, et al. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. *arXiv preprint arXiv:2503.14476*, 2025.

[46] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025.

[47] Yifan Zhang, Yutong Dai, Juntao Tan, Luyu Yang, Rishi Mullur, Thai Hoang, Zhiyuan Hu, James Zhu, Phil Mui, Silvio Savarese, Ran Xu, and Zeyuan Chen. DarwinX: Evolving Agent Harnesses Through Natural Selection. *arXiv preprint arXiv:2608.07545*, 2026.

Appendix

The appendices report every experiment in full, in the order of the body: the simulator pipeline (Appendix A); the 50-task CRMArena-Pro collection and the gate's sample-size study (B); the LLM controller (C); the multi-agent (D) and self-refinement (E) experiments; the continual orchestrator (F); the two further suites (G); the follow-ups (H); the AWR trainer (I); what a real-traffic continuation would need (J); the extended related work (K); the full limitations (L); then every experimental setting (M), a worked logged episode (N), the prompts (O), the estimator derivations (P), the simulator parameters (Q), the per-day tables (R), the reproduction commands (S) and the data license note (T).

A Simulator pipeline: gate, drift, gated retrain

The simulator exercises the whole loop where the truth is known. Tasks have a latent difficulty (easy with probability 0.65, hard 0.35) that the controller never sees; it sees a proxy that is correct 85% of the time. Twelve actions, $\{\textsc{answer}, \textsc{retrieve}, \textsc{verify}, \textsc{escalate}\} \times \{\text{cheap}, \text{mid}, \text{frontier}\}$, each have their own cost, latency and quality tables; episodes have at most four steps, with terminal ops forced on the last. Version v_2, the drift event, adds CACHED_RETRIEVE, near-free evidence that works only on easy tasks; ages the verifier so that VERIFY adds nothing on hard tasks; and reprices the frontier tier by $1.5\times$. Costs are in simulator units.

Setup. 3,000 episodes on v_1 under a uniform behaviour policy; constrained GRPO on replay at budget $B = 0.30$; the candidate gated against the behaviour log. Then drift: v_2 plus a scripted reprice(mid, 1.2); 6,000 fresh v_2 episodes served by the frozen policy with $\epsilon = 0.3$; a retrain warm-started from the frozen policy; the adapted policy gated against the frozen one with the quality bar taken from the frozen policy's v_1 promotion. Ground truth is Monte Carlo over 20,000 fresh tasks with expected success probability. The whole pipeline is repeated over five replicate seeds (seed s draws every collection, training and bootstrap seed from s; Table 19); numbers are mean $\pm$ SE over seeds and the per-seed values are in Table 25.

Results. Figure 7 shows both gates. On v_1 the learned price is $\bar{\lambda} = 0.358 \pm 0.027$. DR puts the candidate at quality 0.884 ± 0.009 (mean 90% CI $[0.851, 0.910]$) and cost 0.308 ± 0.001 (upper bound 0.311); the decision is PROMOTE in 5 of 5 seeds, and the ground truth (0.878 ± 0.001 at 0.306 ± 0.002) lies inside the DR interval in 4 of 5 (in the fifth it is 0.001 below the lower bound). IPS and SNIPS sit low (0.760 ± 0.022 and 0.797 ± 0.014): a uniform behaviour over 12 actions and up to 4 steps gives cumulative ratios in the hundreds against a near-deterministic candidate, and the clip at 50 truncates them. After the drift the frozen policy keeps its quality (0.869 ± 0.001) but its cost rises to 0.413 ± 0.003, over budget, so priced-BWT is -0.047 ± 0.003 (Table 2). The adapted policy is scored at 0.869 ± 0.004 and cost 0.298 ± 0.004 (truth 0.867 ± 0.001 at 0.299 ± 0.004, inside the DR interval in 5 of 5); served, it would recover $+0.039 \pm 0.003$. The gate promotes it in only 2 of 5 seeds. Every revert fails on quality alone, and all three are false alarms: the bar is the DR quality recorded at the v_1 promotion minus ϵ_q, and in those seeds that estimate was high (0.901, 0.883, 0.903 against truths of 0.877–0.878), so a candidate within 0.01 of the frozen policy's quality and inside the budget could not clear it. The served recovery is therefore $+0.015 \pm 0.009$ ($+0.033$ and $+0.042$ where the gate promoted, zero elsewhere). Drift hits cost, not quality: the frozen controller keeps paying for an aged verifier and a repriced frontier, the retrain restores the budget at equal quality, and the quality bar inherited from an optimistic earlier estimate is what keeps it out of service. The continual study finds the same false-alarm mode at larger scale (Appendix F).

Table 2: Simulator drift table, ground truth, mean $\pm$ SE over 5 seeds (J at each seed's own $\bar{\lambda}$). priced-BWT $= J(\text{ii}) - J(\text{i}) = -0.047 \pm 0.003$; recovery $= J(\text{iii}) - J(\text{ii}) = +0.015 \pm 0.009$ as served after the gate, $+0.039 \pm 0.003$ for the adapted policy.

condition	quality	cost	within budget	J
(i) frozen on v_1	0.878 ± 0.001	0.306 ± 0.002	5/5	0.768 ± 0.008
(ii) frozen on v_2	0.869 ± 0.001	0.413 ± 0.003	0/5	0.721 ± 0.011
(iii-a) adapted on v_2 (whether or not promoted)	0.867 ± 0.001	0.299 ± 0.004	5/5	0.760 ± 0.008
(iii) served on v_2 after the gate (2/5 promoted)	0.868 ± 0.001	0.369 ± 0.028	2/5	0.737 ± 0.014

Limitations. The simulator's difficulty structure, success tables and drift event are hand-written; the run demonstrates the mechanics, not effect sizes. Five seeds are enough to show that the second gate's decision depends on the seed, not to estimate its promotion rate. The simulator gate fits its direct model in-sample, which is defensible there because its buckets hold hundreds of transitions; the real-log gates cross-fit.

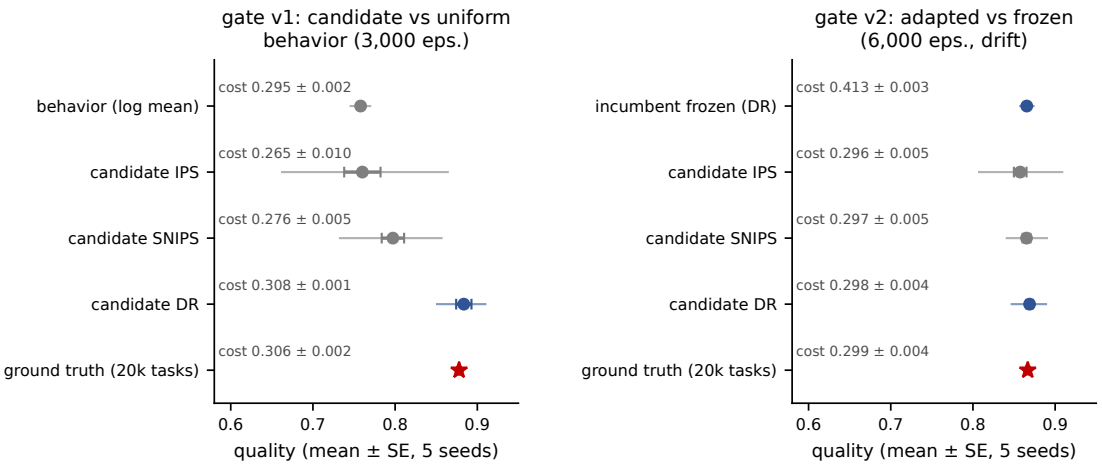


Figure 7: Simulator gates over 5 seeds. Left: the constrained-GRPO candidate scored against 3,000 uniform-behaviour episodes on v_1; right: the adapted policy scored against 6,000 v_2 episodes served by the frozen policy. Points are means over seeds of each estimate with SE bars; thin lines are the mean 90% bootstrap CI; the star is Monte Carlo ground truth (for v_2, the adapted policy whether or not promoted). The gate decides on DR (blue). From `results/sim_seeds/`.

B CRMArena-Pro at 50 tasks: the first collection and the gate's sample-size study

The 300-task results of Section 4.1 are the same code, on the same seeded sample, as the 50-task collection reported here. This first collection is kept in full because it is where the controller memorised and the gate said so, and because the sample-size study of the gate was done on it.

B.1 The counterfactual collection

Setup. 50 B2B tasks (2–3 per category, all 22 categories), each run on all three tiers with a fixed continuation (`CONTINUE` on that tier until the agent answers, `ANSWER` forced at the cap), so that each task's first-step tier is a contextual bandit with a fully observed reward table and the held-out value of any tier policy is an exact lookup. The log holds 150 episodes from 832 model calls; episodes average 5.0 turns and 16 of 150 hit the 12-turn cap. The behaviour is uniform over tiers (propensity $1/3$ at step 0, 1 after). The learned decision is the first-step tier from the 43-dimensional step-0 features. Policies are trained under 5-fold category-stratified cross-validation over task ids (train on 40, evaluate on the held-out 10, seed 0), at fixed $\lambda \in \{2, 5, 10\}$ per dollar with AWR+IPS, and under a budget $B = \$0.10$ with constrained GRPO on the lookup table (learned λ).

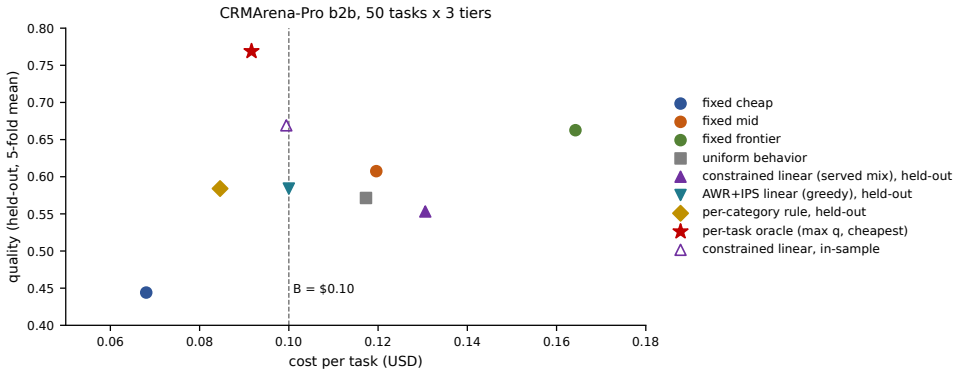


Figure 8: CRMArena-Pro B2B at 50 tasks: held-out quality against cost per task (5-fold means, exact lookups). The per-task oracle beats every fixed arm on both axes; no learned policy does. The open triangle is the constrained policy in-sample (0.669 at \$0.099), the filled one held-out (0.553 at \$0.131).

Results. Table 3 and Figure 8. No learned linear rule clearly beats the best fixed arm held-out. The served AWR policy has lower J than the best fixed arm at every λ (0.350 vs mid 0.368; 0.045 vs cheap 0.104; -0.414 vs cheap

Table 3: Held-out first-step tier policies on CRMArena-Pro B2B at 50 tasks (5-fold means; cost in USD per task; across-fold SD of J in brackets, 0.05–0.3 throughout, as large as the gaps between rows).

policy (held-out)	$\lambda = 2$: J	$\lambda = 5$: J	$\lambda = 10$: J	$B = \$0.10$: quality / cost
AWR+IPS linear, served (stochastic)	0.350 [0.047]	0.045 [0.126]	-0.414 [0.246]	–
AWR+IPS linear, greedy	0.384 [0.076]	0.186 [0.126]	-0.273 [0.272]	–
per-category plug-in rule	0.415 [0.066]	0.161 [0.104]	-0.214 [0.154]	–
constrained cgrpo-lookup (iterate mix)	–	–	–	0.553 / 0.131 (over)
uniform behaviour	0.337 [0.066]	-0.015 [0.164]	-0.601 [0.340]	0.571 / 0.117
fixed cheap	0.308 [0.179]	0.104 [0.227]	-0.236 [0.308]	0.444 / 0.068
fixed mid	0.368 [0.092]	0.010 [0.124]	-0.588 [0.298]	0.608 / 0.120
fixed frontier	0.334 [0.133]	-0.159 [0.317]	-0.980 [0.633]	0.663 / 0.164
per-task oracle	0.639 [0.056]	0.452 [0.080]	0.148 [0.168]	0.769 / 0.092

-0.236), and its greedy version and the per-category rule beat the best arm by at most 0.08, inside the fold SDs. The features memorise: AWR's J at $\lambda = 2$ is 0.475 in-sample against 0.350 held-out; the constrained policy meets the budget in-sample (0.669 at \$0.099) and overspends held-out (0.553 at \$0.131), where the uniform behaviour (0.571 at \$0.117) dominates it. Even the context-free arm mixture tuned to B on 40 tasks costs \$0.115 on the held-out 10. The per-task oracle reaches 0.769 at \$0.092 and beats every arm on both quality and cost: the headroom is per task, not per category, and the missing piece is an observable that predicts which tier a task needs. Repricing events (frontier $\times 1.5$, cheap $\times 3$), with costs recomputed from logged tokens, push the frozen all-50 policy ($\bar{\lambda} = 1.36$ per dollar) over budget by $+\$0.029$ and $+\$0.035$: priced-BWT -0.040 and -0.048, in-sample recovery $+0.069$ and $+0.091$; held-out, the retrained policies still overspend (about \$0.125), and the gate on the repriced realised log REVERTS all three retrains, which the held-out table says is the right call (recovery -0.012 for frontier $\times 1.5$).

Limitations. Fifty tasks, one organisation, one seed; one task moves a fold's quality by 0.1. Only the first-step tier is learned, because the continuation was fixed at collection. The log holds one harness version, so version drift cannot be measured on it. Prompt caching was not used, so the roughly 5k-token system prompt is paid on every turn.

B.2 The gate's sample-size study at 50 tasks

Setup. The candidate is the constrained policy trained on all 50 tasks (exact value: quality 0.665, cost \$0.0987, within $B+\text{tol} = \$0.105$). The realised behaviour log is one uniformly drawn arm per task (50 episodes). The gate (cross-fitted DR, $\epsilon_q = 0.03$) is run once, then over 200 independent redraws of the realised log with a 200-resample bootstrap in each.

Results. On the single realised log DR estimates quality 0.740 $[0.627, 0.855]$ and cost \$0.0960 $[0.0750, 0.1200]$; the quality criterion passes and the cost criterion fails ($0.1200 > 0.105$): REVERT. On the full counterfactual log all three estimators reduce to the exact value (the weights over a task's three arms sum to 3) and the interval is pure task-sampling uncertainty from 50 tasks; the cost upper bound is still \$0.1122 and the decision is still REVERT. Over the 200 redraws (Table 4) DR halves IPS's quality RMSE (0.048 vs 0.094) with 99% coverage of its 90% interval, but its cost interval is \$0.046 wide on average, so the candidate, whose exact cost is \$0.0013 under $B+\text{tol}$, is PROMOTED in only $12/200 = 6\%$ of redraws. The gate errs on the safe side; buying power means more tasks or a wider tolerance. With interval width scaling as $1/\sqrt{n}$, certifying a \$0.005 margin needs an interval near \$0.01, roughly twenty times the tasks, about 1,000. Section 4.2 reports how this projection fared at 150 and 300 tasks.

Table 4: Estimators over 200 redraws of the realised CRMArena-Pro log ($n = 50$, one arm per task). Cross-fitted direct model. PROMOTE in 12/200 redraws (6%).

metric	estimator	bias	RMSE	mean 90% CI width	CI covers exact
quality	IPS	$+0.011$	0.094	0.338	94%
quality	SNIPS	$+0.001$	0.048	0.238	100%
quality	DR	$+0.004$	0.048	0.256	99%
cost	IPS	$+0.0001$	0.0114	0.0414	92%
cost	SNIPS	-0.0009	0.0102	0.0381	92%
cost	DR	-0.0018	0.0114	0.0461	92%

Limitations. An in-sample direct model covers 96%/94% with a -0.006 quality bias, a difference inside the noise at $n = 50$; the redraw study reports bias and coverage empirically rather than assuming them.

C LLM controller: GRPO with LoRA on a 1.7B model

The linear controller of Section 3.2 can be replaced by a small language model that reads the state as text and emits the action as a tag. This experiment trains such a controller with GRPO under the same learned price and asks whether it finds the budgeted optimum.

Setup. The controller is Qwen3-1.7B [30] with a LoRA adapter [11] ($r = 16$ on the attention and MLP projections, bf16), trained with GRPO. The agent models stay frozen; the tiers are the arms. The prompt shows the difficulty proxy, the state, per-tier prices, the budget and the legal actions, and the completion must be exactly one tag `<action>TIER OP</action>`; anything else is invalid (reward -1, cost booked as the cheapest safe action). After the first action a fixed continuation answers on the same tier; the reward is $q - \lambda C$ with λ learned by the same damped dual ascent, once per step from the batch mean cost. Each step generates 64 completions (8 prompts $\times$ 8 generations in phases 1 and 1b; 4 $\times$ 16 in 1c), of length 16, at learning rate 10^{-5}, over 2,000 seeded simulator tasks with the legal-action order shuffled per task (with a fixed order the base model only ever sampled one or two arms). Three runs on the simulator, each run once on one A100-40GB: `phase1` (λ learning rate 0.1, k_p 2.0, β_{KL} 0.02, 300 steps); `phase1b` (a slower dual step: learning rate 0.02, k_p 0.5, EMA 0.95; β_{KL} 0.05; entropy 0.01); `phase1c` (150 SFT steps on per-difficulty oracle labels at a price shown in the prompt and drawn from $\{0.0, 0.1, \ldots, 2.0\}$, the adapter merged so that the KL reference is the SFT policy, then 200 GRPO steps with 16 generations, temperature 1.2, $\lambda_0 = 0.5$). A fourth run (`crm_phase2`) applies the same recipe to CRMArena-Pro fold 0 in lookup mode. Per-run settings are in Appendix M.

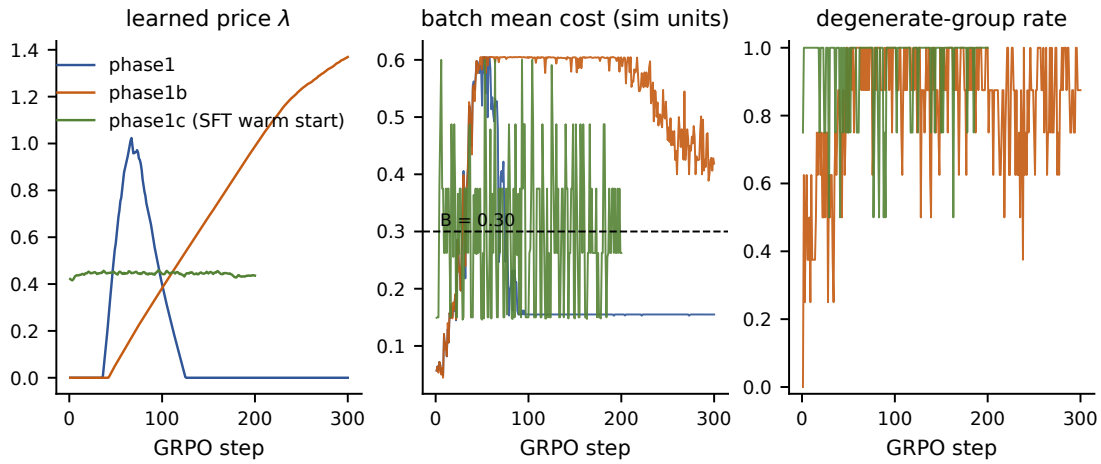


Figure 9: GRPO training logs. `phase1`'s price orbits once and decays to 0 as the policy collapses onto `mid RETRIEVE`; `phase1b`'s price is still rising at step 300 while cost stays above budget; `phase1c`'s price settles at $\lambda \approx 0.44$ with the batch cost alternating between two deterministic rules. Degenerate groups (all completions identical, hence zero advantage) dominate every run.

Table 5: LLM controller on simulator v_1 (2,000 fresh tasks, seed 777; budget $B = 0.30$; the policy's first action then `ANSWER` on the same tier). "Within" means cost $\leq B + 0.02$.

policy	quality	cost	within	invalid
Qwen3-1.7B zero-shot (greedy, no adapter)	0.586	0.048	yes	18.9%
`phase1`, steps 100 / 200 / 300	0.748–0.749	0.155	yes	0%
`phase1b`, step 100	0.885	0.603	no	0%
`phase1b`, step 300	0.832	0.434	no	0.1%
`phase1c`: SFT only, and GRPO steps 100 and 200 (all identical)	0.848	0.330	no	0%
linear controller (`policy_v1_sim.json`), first action only	0.573	0.035	yes	0%
fixed `ANSWER` on cheap / mid / frontier	0.522 / 0.688 / 0.820	0.020 / 0.100 / 0.400	yes / yes / no	0%
best deterministic per-proxy rule (easy: mid ESCALATE; hard: frontier ANSWER)	0.814	0.250	yes	–
best randomised policy at cost exactly B (LP)	0.835	0.300	yes	–
derived: `phase1c` served with the price drawn as in training's last 100 steps	0.837	0.305	yes	–

Results. Table 5 and Figure 9. GRPO alone never found the budgeted optimum, for two reasons that the logs make visible. *Group degeneracy*: within a prompt group the completions are almost always identical, 83.5% of groups on average in `phase1b`, which gives zero group-relative advantage, so the only learning signal is λ moving the mean reward between steps, and the policy jumps between near-deterministic arms instead of learning the per-proxy split. *Primal–dual orbit*: in `phase1` the policy swings to `frontier RETRIEVE` (cost 0.60), λ climbs to 1.02 by step 67, the policy collapses onto `mid RETRIEVE` (0.155), and λ decays to 0 with nothing to pull it back; in `phase1b` λ is still rising at step 300 (1.37) and the policy has not reached the budget. Mixing is not the bottleneck: a deterministic per-proxy rule reaches 0.814 at 0.250 and randomisation adds only 0.021; what the runs failed to do is condition on the proxy. In `phase1c` the SFT stage reached 100% held-out agreement with the oracle labels (1,000 prompts, 0% invalid) and splits by difficulty where the oracle does; GRPO then left the policy unchanged on all 2,000 evaluation tasks (the degenerate-group rate averaged 93% over steps 1–100 and 97% over 101–200, even after the rollout temperature was raised from 1.2 to 1.5 at step 20). GRPO's contribution is the price: λ settles at 0.44 (mean 0.443 over the last 100 steps), which is the slope at which the hard-proxy action flips, and at that price the SFT policy ($J = 0.702$) ties the oracle rule (0.703) and the LP (0.702). No single greedy policy meets $B = 0.30$: at any one shown price the policy is a deterministic rule costing 0.33 or 0.15; training met the budget by alternating the shown price over time (batch cost 0.301 and 0.296 over the two halves), and serving can reach it by drawing the shown price, the price-conditioned analogue of iterate averaging: 0.837 at 0.305, derived from the two greedy rules rather than decoded end to end. On CRMArena-Pro fold 0 the SFT policy ignores the shown price (the same per-category rule at every λ, about \$0.075, under budget) and GRPO again leaves it unchanged.

The platform controls. The `phase1b` failure was reproduced on three stacks, TRL 1.15, verl 0.9.1 on Ray with vLLM, and NeMo-RL, with the same model, adapter, data and dual-ascent settings, and then run again with the price held fixed at $\lambda = 0.44$ (Appendix M, Table 23; `results/llm/backends/COMPARISON.md`). With the price moving, every stack reproduces the collapse. With the price fixed, every stack learns the per-proxy split, and the three agree within 0.001 in priced value (J 0.681 / 0.682 / 0.682 at cost 0.333 / 0.334 / 0.333); a fixed-price TRL control without dynamic sampling also splits (J 0.679, within budget), and DAPO's dynamic sampling [45], which drops the degenerate groups from the batch, sharpens the hard-task split from 53% to 98–100% frontier for $+0.065$ cost. The cause of the failure was the moving price, not the sampler or the platform.

Limitations. Each GPU run ran once; there is no seed variance. The phase-1 table truncates the linear policy's continuation (its first action is `cheap RETRIEVE` because it was trained to keep going; with its own continuation it scored 0.876 at 0.301 in the run that produced that file), so its row is not its real performance. Showing λ rounded to 0.1 puts $\bar{\lambda} = 0.443$ on the over-budget side of the flip point.

D Process-verified multi-agent orchestration

This experiment asks whether a planner/worker/verifier decomposition of the CRM task clears the compute-matched single-agent bar that MAS-Orchestra [15] reports, and whether per-step verification of the kind MAS-ProVe [40] studies carries information on this log.

Setup. A planner (cheap tier) decomposes a CRM request into 1–3 sub-queries; a data worker (mid) runs one sub-query at a time through the harness's own step path; a verifier (cheap) checks a sub-result against its goal (a structural pre-check fails at \$0 on query errors); the answer is the harness's forced `ANSWER@mid`. After every worker step an orchestration policy chooses $\{\textsc{verify}, \textsc{skip_verify}, \textsc{replan}, \textsc{answer}\} \times$ tier; a fail retries the sub-query once; at most 8 role calls per episode. Fixed modes `always_verify` and `never_verify`, plus a uniform exploration mode with logged propensities, were run on the same 50 B2B tasks as the counterfactual log: 130 episodes (50 + 50 + 30). An orchestration reward model (ORM: logistic regression and a 16-unit MLP over process features) is evaluated out of fold over tasks.

Results. The headline is negative (Table 6): the decomposition scores below the single mid-tier agent (0.480 vs 0.608) at lower cost and is dominated on the priced frontier at both prices (by mid at $\lambda = 2$, by cheap at $\lambda = 5$); paired on the 50 tasks the single mid agent wins 11 and loses 5. The compute-matched single-agent bar is not cleared. Three mechanisms are visible in the log. The plan is fixed up front by a cheap model with no data in hand, while the single agent picks each query after seeing the last result. `never_verify` gives the strong model about 2.7 worker turns and a forced answer, against the single agent's about 5. Under `always_verify`, 47 of 50 episodes hit the 8-call cap, because 65% of verifier calls fail and each fail spends a retry. Verification bought $+1$ task for $+10\%$ cost, and the paired table says that is noise (6 flips each way, all through a fail and a retry). The verifier is an alarm, not a judge: it flags 24 of 26

Table 6: Decomposition against the single ReAct agent on the same 50 CRMArena-Pro B2B tasks (single-agent rows are exact lookups of the counterfactual log). The six confidentiality tasks score 0 for every system, so the ceiling is 0.88.

system	accuracy / quality	cost / task	$J(\lambda = 2)$	$J(\lambda = 5)$
single agent, fixed cheap	0.444	\$0.0680	0.308	0.104
single agent, fixed mid	0.608	\$0.1196	0.368	0.010
single agent, fixed frontier	0.663	\$0.1642	0.334	-0.159
single agent, per-task oracle (max quality, cheapest)	0.769	\$0.0916	0.586	0.311
MAS `always_verify`	0.480	\$0.0906	0.299	0.027
MAS `never_verify`	0.461	\$0.0821	0.297	0.051
MAS `controller:uniform` (30 tasks, exploration)	0.535	\$0.0758	0.384	0.156

Paired, same 50 tasks: `always_verify` vs `never_verify`: 6 wrong→right, 5 right→wrong, net $+1$ ($\Delta q = +0.019 \pm 0.067$ SE, $+\$0.0086$). `always_verify` vs single mid: both right 19, MAS right / mid wrong 5, MAS wrong / mid right 11, both wrong 15 (net -6).

Table 7: Orchestration reward model, out of fold over tasks (130 episodes, base rate $P(\text{correct}) = 0.485$, base-rate Brier 0.250).

model	features	AUROC	Brier	ECE (5 bins)
logreg	process features (default)	0.469	0.281	0.114
logreg	process − verifier features	0.451	0.280	0.140
logreg	process + category one-hot	0.831	0.165	0.040
mlp	process features (default)	0.456	0.263	0.053
mlp	process + category one-hot	0.831	0.164	0.035
–	category alone, no process feature	0.821	0.166	–

wrong episodes ($P(\text{flag} \mid \text{wrong}) = 0.92$, silent failures $2/50$) and 23 of 24 correct ones ($P(\text{flag} \mid \text{correct}) = 0.96$), so $P(\text{wrong} \mid \text{flagged}) = 0.51$ against a base rate of 0.52. The ORM (Table 7) finds that process features carry no out-of-fold signal (AUROC 0.469, Brier above the base rate) and that the task category carries most of it (0.821 alone; 0.831 with process features); the cross-validation is over tasks, so this is category-level difficulty transferring to unseen tasks, resting on 2–3 training tasks per category. The priced-controller hook was exercised, not evaluated: AWR on the 30-episode ORM-relabelled exploration log changes nothing detectable per decision (per-decision SNIPS 0.563 $[0.373, 0.701]$ against the behaviour's 0.567, ESS 55 of 64), and the episode-level estimate has ESS 7 of 30.

Limitations. The verifier is a cheap model of the same family checking a mid model; there is no human process label. The 8-call cap interacts with a high-fail verifier, so a higher cap or a better verifier is a different system; the ablation of Appendix F.2 varies exactly those two things. Sub-queries run sequentially in one growing context.

E Priced Socratic self-refinement

Test-time refinement is a purchase: a probe, some re-solving, a rewrite. This experiment asks whether a priced gate can decide when to buy it, and whether buying it beats buying a better tier at the first turn.

Setup. One Socratic self-refinement round [32] on a CRMArena-Pro answer: a cheap probe decomposes the answer into 1–3 claims with self-reported confidences (about \$0.003, so a gate can look before it pays); each claim is resolved by $k = 2$ worker mini-episodes on the chosen tier (≤ 3 turns, with the SOQL run on the live organisation); a rewrite on that tier produces the refined answer; the benchmark's evaluator scores it. The initial answers are the cheap-tier episodes of the counterfactual log (already scored and paid). The run logged a tree, a refinement on every tier for every task, so every policy over $\{\text{STOP}, \text{REFINE@cheap}, \text{REFINE@mid}\}$ is an exact lookup: `never`, `always_1:t`, `uncertainty_gated:`τ`:t` (τ swept 0.3–0.9 on the minimum claim confidence), and a softmax-linear priced controller over 36 features trained by exact policy gradient on the tree (budget $B = \$0.10$ with learned λ, or fixed $\lambda \in \{2, 5\}$; 5-fold CV over tasks). Of 42 tasks probed, 41 completed on both tiers. One task is 0.024 of quality.

Table 8: Refinement harm after one round (41 tasks). "fix" is wrong→right out of initially wrong, "harm" right→wrong out of initially correct.

tier	changed	fix	harm	mean Δq	round cost	worker turns
cheap	85% (35/41)	13% (3/23)	28% (5/18)	-0.049	\$0.0962	14.7
mid	44% (18/41)	13% (3/23)	6% (1/18)	$+0.049$	\$0.2380	13.9

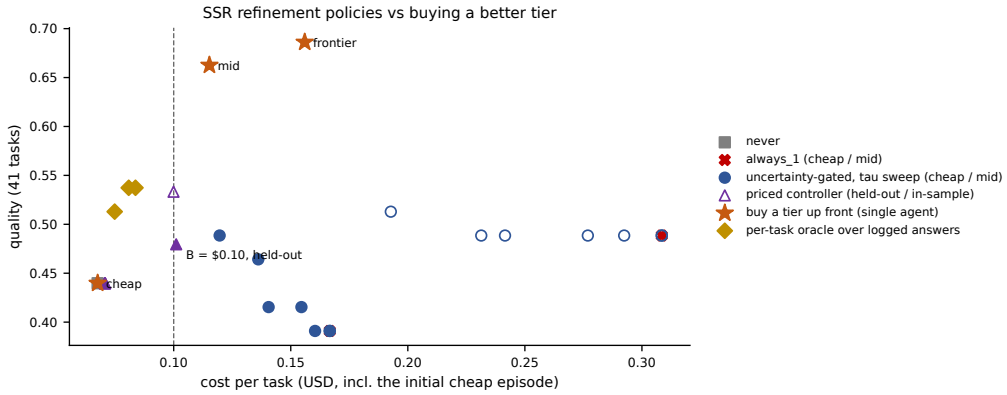


Figure 10: Self-refinement policies on the logged tree (41 tasks; cost includes the initial cheap episode), from `frontier.csv`. Open circles are mid-tier gates, filled are cheap-tier. The single-agent mid arm at \$0.115 dominates every refinement policy.

Results. The probe's confidence is weakly informative: AUROC 0.65 (90% CI 0.51–0.79, $n = 41$) of $1 -$ min claim confidence for predicting a wrong initial answer. The decomposer is pessimistic about everything (minimum claim confidence never above 0.75, below 0.4 on 26 of 41 tasks whether right or wrong), the re-solve signal (AUROC 0.67 cheap, 0.55 mid) is no better than the free one, and 22% of mid worker samples are "None". Refinement harms as often as it helps (Table 8): a cheap round fixes 3 of 23 and breaks 5 of 18 ($\Delta q = -0.049 \pm 0.069$), a mid round fixes 3 and breaks 1 ($+0.049 \pm 0.049$), both within one SE of zero. Policies (Figure 10): `never` 0.440 ± 0.078 at \$0.068; `always_1:cheap` 0.391 at \$0.167; `always_1:mid` 0.489 at \$0.309; the best fixed cheap gate ($\tau = 0.3$) 0.489 at \$0.120; the budgeted controller 0.480 ± 0.077 at \$0.101 held-out (paired $+0.040 \pm 0.033$ over `never`, refining 20% of tasks, $\bar{\lambda} = 0.09$ per dollar because it sits under budget; 0.533 at \$0.100 in-sample); the fixed-λ controllers learn `never`. Even the clairvoyant choice among the three logged answers per task reaches only 0.537 at \$0.084. Of the three hypotheses, refine-where-confidence-is-low is not established; always-refine-is-dominated-under-a-budget is supported, on the cheap tier by `never` itself; a-priced-gate-reaches-the-frontier holds only in the weak sense that the held-out controller sits on the frontier by learning to refine rarely. The comparison that matters is refine-the-cheap-answer against buy-mid-up-front, and buying the tier wins by a wide margin: the single-agent mid arm on the same 41 tasks scores 0.663 ± 0.074 at \$0.115; the best cheap-refinement policy at that price is -0.174 ± 0.085 paired for $+$\$0.004; `always_1:cheap` is -0.272 ± 0.099; refining on mid at $2.7\times$ the cost is still -0.174 ± 0.092; and the frontier tier up front (0.686 at \$0.156) beats every refinement policy by 0.17–0.30. The sub-questions need the same schema reasoning that made the cheap agent fail, and the workers inherit it.

Limitations. One round, 41 tasks, one initial arm; the τ sweep is tuned on the tasks it is evaluated on. The in-sample controller row shows what 22 category one-hots memorise on 41 tasks. Agreement with $k = 2$ is weak, and no sampling parameters are sent.

F The continual orchestrator: full results

Section 4.3 states the findings; this appendix gives the harness, the simulator and its calibration, the pre-registered stages, the day loop and comparators, every result table, the mild-schedule sensitivity run and the verifier ablation.

Harness definition. The multi-agent structure of Appendix D is expressed as a typed harness in the terms of SkillOrchestra and EvoHarnessBench: an agent pool of typed specialists (agent id, tier, mode, stage) with modes planner, data-worker, verifier and answer; tools the orchestrator can `call@`; skills it can `engage@`; and a competence handbook over the 22 task categories, the axis the real run found carries the signal. The harness $H_t = \{\text{tools}, \text{skills}, \text{agents}\}$ grows monotonically by stages, $H_1 \subseteq H_2 \subseteq \cdots$, plus a price axis. The structure at each stage is frozen and the *policy* at the decision point is what adapts, so the question is EvoHarnessBench's turned around: when H expands under an orchestrator that keeps learning, what happens to the policy's backward and forward transfer, and does the gate keep it safe without making it too slow to matter? There are eighteen actions, `delegate@data-1..4`, `verify@verify-1,2`, `replan@plan-1..3`, `answer@answer-1..4`, `call@schema_lookup`, `call@aggregate_calc`, `engage@soql_aggregate`, `engage@date_window` and `engage@decompose_fine`, and 88 features: the real orchestrator's 66, computed by its own feature function on a view of the simulated state, plus availability flags, tools called, skills engaged, stage and a last-fail flag. Unavailable capabilities have fixed zero feature slots and zero logits, so a policy trained at stage g keeps

its weights when stage $g+1$ arrives.

Simulator and calibration. Days of traffic under several harness changes are beyond what the real collection covers, so the logged process is re-expressed as a seeded simulator whose decision points, legal-set rules and feature layout are the real orchestrator's. Every parameter is either measured from the two committed logs (130 multi-agent episodes, 150 single-agent episodes), with its count, or marked assumed (Appendix Q). Measured: the plan-length distribution (4/22/104 of 130), worker step kinds per tier, the retry re-draw, the verifier's per-call fail rates by step kind (72 of 124 on rows, 23 of 24 on empty, 12 of 12 structural at \$0) and its episode-level $P(\text{flag} \mid \text{wrong}) = 0.92$ and $P(\text{flag} \mid \text{correct}) = 0.96$, token means per role and tier, per-category accuracy, and the configuration. Assumed: whether a rows step is the *right* rows ($P(\text{good} \mid \text{rows}) = 0.60/0.75/0.85$ by tier), the logit slope from plan coverage to the answer (1.5, centred so that `never_verify` reproduces the category base rates), a task-level logit noise (sd 0.4), and everything that arrives after stage 0. On the stage-0 harness the two fixed modes reproduce the real run's process numbers (Table 9).

Table 9: Simulator calibration check: stage-0 harness, 1,000 simulated tasks per mode against the real run's 50 tasks per mode (sim / measured).

mode	accuracy	cost/task	role calls	verifier calls	cap hits	$P(\text{flag} \mid \text{wrong})$	$P(\text{flag} \mid \text{correct})$
`always_verify`	0.446 / 0.480	\$0.0894 / \$0.0906	7.87 / 7.88	2.94 / 2.94	95% / 94%	0.95 / 0.92	0.93 / 0.96
`never_verify`	0.459 / 0.460	\$0.0827 / \$0.0821	4.75 / 4.74	0 / 0	0% / 0%	–	–

Pre-registered stages. Both schedules were written down before the first full run and not changed afterwards (Table 10). The `mild` schedule keeps the days and axes with smaller effects: reprice mid $\times 0.8$ / frontier $\times 1.2$, `verify-2` at $P(\text{flag} \mid \text{correct}) = 0.60$, `data-4` at rows 0.78 / good 0.75, `answer-4` mid-grade on 3 categories, half the tool and skill bonuses, planner v2 at 1–3 sub-queries.

Table 10: The pre-registered default schedule: 40 days, events on days 9 / 17 / 25 / 33.

stage	day	axis	what changed
g0 baseline	1	–	the real run's pool: plan-1, data-1..3, verify-1, answer-1..3; list prices
g1 reprice	9	price	mid $\times 0.5$, frontier $\times 1.5$
g2 agents	17	agents	+verify-2 (a better verifier: $P(\text{flag} \mid \text{correct})$ 0.96 $\rightarrow$ 0.30 at the same recall), +data-4 (database specialist, cheap), +answer-4 (reporting specialist, cheap)
g3 tools	25	tools	+schema_lookup (the next worker step cannot error), +aggregate_calc
g4 skills	33	skills	+soql_aggregate, date_window, decompose_fine; the planner now decomposes into 1–4 sub-queries

The day loop and comparators. Each day is 96 seeded tasks, the same for every arm. The continual arm serves its incumbent with $\epsilon = 0.1$ and logs propensities; a learner trains every day by constrained GRPO on replay from the last 5 days of logged states (60 iterations, batch 16, group 6; λ learned against $B = \$0.090$ per task; iterate averaging), warm-started from itself; the gate (sequential DR with 200 bootstrap resamples, $\epsilon_q = 0.05$, tol $= \$0.005$) scores the learner against the incumbent on the log served since the last harness event (up to 5 days), with the budget net of the policy-independent prefix (planner plus first worker call, \$0.022 at list prices). An AWR+IPS variant (fixed $\lambda = 2$, 200 epochs) runs the same loop. Baselines on the same traffic: `frozen` (the learner as of day 5, served without ϵ, restricted to stage-0 actions); `so_static`, a SkillOrchestra-style router whose handbook is the literal rule competence$[(a, \text{category})] - \lambda_c \cdot \overline{\text{cost}}[a]$ with cost tie-break, learned from the warm-up exploration traces (days 1–4) with λ_c chosen on day 5 by episode-level SN-IPS and never updated (new capabilities scored at the handbook's prior); `so_relearn`, the same router re-fitted after every stage (3 days served with $\epsilon = 0.3$, then a refit and re-tune, the lag counted); `always_verify`; `never_verify`; and a per-stage `oracle` (constrained GRPO on 200 fresh exploration episodes of that stage, 300 iterations, no gate: a reference, not a proven optimum). Held-out evaluation uses 150 fixed tasks per stage, scored by simulator expected quality and never seen by the gate; $J_g = \text{quality} - \bar{\lambda}_g \cdot \text{cost}$ with $\bar{\lambda}_g$ the stage oracle's learned price (1.25 / 0.85 / 0.81 / 0.66 / 0.73 per dollar for g0–g4), at which the budgeted optimum maximises the Lagrangian, so that J_g has a ceiling and regret is well defined. Metrics follow Eq. (11); in addition, hit_j is the event-day policy's J on stage j minus on stage $j-1$, $\text{recovery}_j = R_{j,j} - R_{j-1,j}$, days-to-recover is the first day of the stage within 0.02 of the oracle, stationary J the mean over the stage's last four days, and cumulative regret the

sum over days 6–40 of the oracle's J minus the serving policy's on the day's stage. One seed, on CPU; five seeds for the arms covered by the ablation below.

F.1 Results

(1) The fixed mode `never_verify` wins the whole stream; against the learned alternatives the continual orchestrator wins on every metric. Cumulative regret over days 6–40 (Table 12, Figure 4): `never_verify` 0.56, continual (cgrpo, gated) 1.26, `always_verify` 1.99, frozen 2.60, router re-learned per stage 3.03, static router 3.46, continual AWR 3.90. `never_verify`, the all-mid fixed mode, is at the oracle on g1 (0.466 vs 0.460) and within 0.02–0.04 on g2–g4 without learning anything: after the mid repricing, running every sub-query and the answer on the half-price mid tier is the budgeted optimum or near it in this simulator. The continual arm ends every stage within 0.001–0.014 of the oracle and is the only learned arm that recovers at all (days to recover 6 / 6 / 1 / 1); the frozen controller, the static router and the re-learned router never come within 0.02 of the oracle on any stage. The continual arm beats `never_verify` only in the stationary periods of the capability-adding stages (g2 0.429 vs 0.423, g3 0.453 vs 0.434, g4 0.448 vs 0.428) and loses that back during the recovery days. Over 5 seeds of the same configuration (variant A of the ablation below; mean ± SE), the continual-minus-`never_verify` stationary J is -0.050 ± 0.011 (g0), -0.009 ± 0.002 (g1), -0.014 ± 0.007 (g2), $+0.011 \pm 0.010$ (g3) and $+0.031 \pm 0.007$ (g4), and the whole-stream regret is continual 1.06 ± 0.21, `never_verify` 0.74 ± 0.08, `always_verify` 1.97 ± 0.09: the fixed mode's whole-stream win and the learner's margin at the last stage are both outside seed noise; the g2–g3 margins are not. The honest answer to "does the learner beat the alternatives" is that it beats every design-time learner, and the thing that beats it is knowing the right fixed policy in advance, which the growing pool makes less likely with every stage: the fixed mode's gap to the oracle grows from -0.006 at g1 to -0.038 at g4.

Table 11: Per-event transfer on the default schedule (held-out J of the policy in service; oracle J per stage g1–g4: 0.460 / 0.445 / 0.458 / 0.466; uniform 0.351 / 0.324 / 0.317 / 0.318). Stationary J per stage and BWT per stage are in Appendix R and Figure 12.

arm	hit at event g1/g2/g3/g4	FWT g1/g2/g3/g4	days to recover	gap to oracle at stage end g1/g2/g3/g4
continual (cgrpo, gated)	$+.048/-.050/+.012/-.008$	$-.020/+.085/+.130/+.128$	6/6/1/1	$-.001/-.010/-.004/-.014$
continual (AWR+IPS, gated)	$+.071/-.019/-.007/-.004$	$+.012/+.016/+.014/+.006$	–/–/–/–	$-.102/-.107/-.130/-.155$
frozen (days 1–5)	$+.067/-.041/+.004/-.024$	$+.060/+.046/+.057/+.032$	–/–/–/–	$-.050/-.075/-.084/-.116$
static router	$+.053/-.033/-.002/-.033$	$+.034/+.028/+.032/-.002$	–/–/–/–	$-.075/-.093/-.109/-.150$
router re-learned per stage	$+.053/-.041/+.015/-.026$	$+.034/+.024/+.066/+.002$	–/–/–/–	$-.071/-.076/-.112/-.103$
`always_verify`	$+.097/-.037/+.008/-.021$	$+.075/+.065/+.080/+.057$	–/–/–/–	$-.034/-.056/-.062/-.090$
`never_verify`	$+.098/-.043/+.011/-.006$	$+.115/+.099/+.117/+.109$	1/–/–/–	$+.006/-.022/-.024/-.038$

Table 12: Whole-stream summary, default schedule (mild schedule in parentheses).

arm	mean BWT	mean FWT	regret, days 6–40
continual (cgrpo on replay, gated, learned λ)	$+0.031$ ($+0.035$)	$+0.081$ ($+0.075$)	1.262 (1.100)
continual (AWR+IPS, gated, $\lambda = 2$)	-0.006 (-0.005)	$+0.012$ ($+0.002$)	3.896 (3.668)
frozen (trained days 1–5)	0	$+0.049$ ($+0.053$)	2.597 (1.970)
SkillOrchestra-style static router	0	$+0.023$ ($+0.025$)	3.459 (2.902)
SkillOrchestra-style router, re-learned per stage	$+0.019$ ($+0.017$)	$+0.032$ ($+0.014$)	3.029 (3.057)
`always_verify`	0	$+0.069$ ($+0.062$)	1.988 (1.731)
`never_verify`	0	$+0.110$ ($+0.100$)	0.559 (0.394)

(2) The whole-stream headline is a cold-start artefact: the learner crosses `never_verify` around day 22–25 and wins every stage window after that, while `never_verify` falls further below the oracle at each capability stage. Figure 5 in the body. Served J, continual minus `never_verify`, by 8-day stage window: g0 -0.073, g1 -0.070, g2 -0.021, g3 $+0.024$, g4 $+0.035$; on the held-out sets -0.085, -0.087, -0.004, $+0.016$, $+0.019$. The served gap is positive from day 24 on and stays positive. Cumulative held-out regret to the oracle restricted to days 17–40 is 0.42 (continual) against 0.68 (`never_verify`); restricted to days 25–40 it is 0.22 against 0.50, and every other arm is further behind (frozen 1.60, re-learned router 1.74, static 2.07, AWR 2.14 from day 25). The learner's served J includes its $\epsilon = 0.1$ exploration mass, a tax the fixed policy does not pay, and it still wins after the crossover. The whole deficit in the whole-stream number is the 12 days on the uniform policy before the first promotion (finding 3) plus the repricing recovery (days 9–13, where the half-price mid tier makes `never_verify` the optimum before the learner has moved): `never_verify` is above the reference oracle at g0 and g1 ($+0.023$, $+0.006$ on the held-out sets at stage end) and falls

to -0.022, -0.024, -0.038 at g2–g4 as capabilities it cannot use arrive. The post-crossover margin at g4 holds over 5 seeds ($+0.031 \pm 0.007$, above); the stage-2+ capabilities are assumed parameters (Appendix Q), so its size is the simulator's.

(3) The gate's price is larger than its benefit on this stream: 32 of 40 candidates reverted, 9 rightly and 23 as false alarms, first promotion on day 13. Figure 11 and Table 13. Every one of the 9 correct REVERTS was a budget violation (candidate true cost \$0.095–0.101 against $B+\text{tol} = \$0.095$) and no promoted candidate was bad. The DR cost estimate is the criterion that works: RMSE \$0.006 against the simulator's truth, and the set of candidates it stopped is exactly the set of over-budget candidates. The quality criterion produces the false alarms: 18 of the 32 REVERTS failed on quality alone with the candidate acceptable. At $n \approx 500$–$1{,}000$ logged episodes the DR quality 90% interval is 0.20 wide and covers the truth 63% of the time, so a candidate whose lower bound must reach the incumbent's point estimate minus 0.05 passes only when it is visibly better. During days 1–12 the served policy was therefore the uniform ϵ-policy while the learner was already good: the frozen arm, which is the learner's day-5 snapshot, scores 0.411 on g1 against the served incumbent's 0.247 on day 9. This is the same sample-size finding as Appendix B.2: the cost side is certifiable at about 500 episodes a day, the quality side is not.

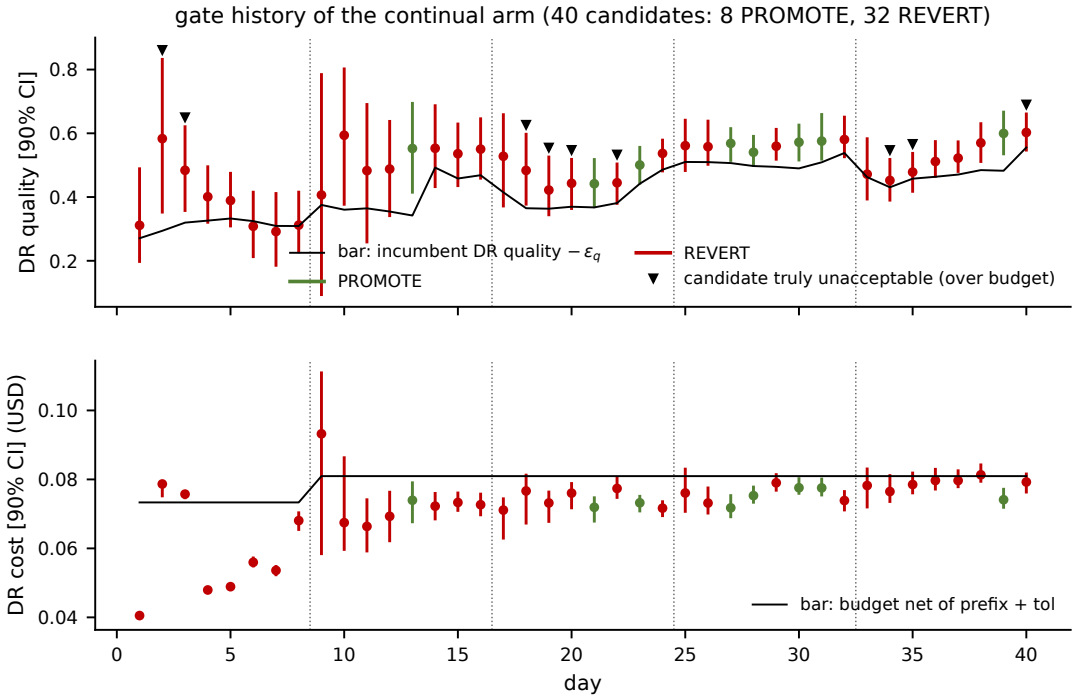


Figure 11: Gate history of the continual arm (from `gates.jsonl`). Top: DR quality with its 90% CI against the bar (incumbent's DR quality minus ϵ_q); bottom: DR cost against the budget net of the prefix plus tolerance. Green is PROMOTE, red REVERT; triangles mark candidates the simulator says were truly over budget. The cost criterion catches every bad candidate; the quality criterion rejects acceptable ones until they are visibly better.

(4) The AWR+IPS variant is the cautionary row. It was promoted 14 times and is the worst learned arm (stationary J 0.32–0.36) because the gate's sequential DR over-estimated its quality by $+0.68$ on average (coverage 6%). AWR's exponentiated inverse-propensity weights move the policy far from the behaviour in one full-batch step; the cumulative ratios (clipped at 50) then carry the estimate with a per-decision ESS of 25% and a bias the clip does not remove. The per-decision estimators' bias on the same candidates is $+0.18$, better but not a safe bar. The cgrpo candidates, trained on rollouts sampled from the ϵ-mixed served policy, stay close to the behaviour (ESS 54%) and the same estimator is nearly unbiased on them (-0.005). The gate is only as good as the overlap between candidate and behaviour, and a trainer that preserves that overlap is part of the safety story.

(5) Orchestrator-level BWT is bounded and mostly positive. Figure 12. The continual arm's BWT per stage is $+0.111$ (g0), $+0.023$ (g1), $+0.009$ (g2), -0.019 (g3); the mild schedule gives $+0.120$, $+0.026$, $+0.002$, -0.009. Later training did not undo earlier behaviour because the harness only grows: a capability that arrives at stage g has a zero feature slot and a zero logit before it, and the stage-g weights are still what runs where it is not legal. Contrast

Table 13: Gate history and estimator accuracy against simulator truth (the candidate's true held-out value on the current stage), days after warm-up (35 gates, mean $n = 370$ episodes per gate).

	continual (cgrpo on replay)	continual (AWR+IPS)
candidates: PROMOTE / REVERT	8 / 32	14 / 26
REVERT right (candidate bad) / false alarm	9 / 23	0 / 26
PROMOTE right / missed (candidate bad)	8 / 0	14 / 0
mean per-decision ESS / n	0.54	0.25
mean DR quality CI width / cost CI width	0.200 / \$0.0079	0.653 / \$0.0144
DR quality: bias / RMSE / CI covers truth	-0.005 / 0.067 / 63%	$+0.676$ / 0.809 / 6%
DR cost: bias / RMSE / CI covers truth	$+0.0003$ / \$0.0061 / 54%	$+0.0000$ / \$0.0056 / 69%
per-decision SNIPS quality: bias / RMSE	-0.020 / 0.053	$+0.178$ / 0.187

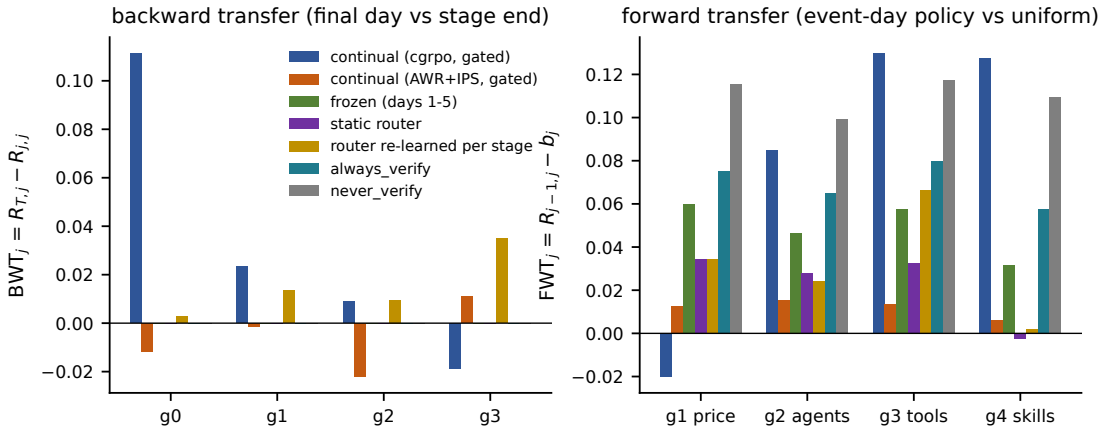


Figure 12: Orchestrator-level backward and forward transfer per stage, Eq. (11), computed from `days.jsonl`. The fixed arms have BWT 0 by construction. The continual arm's large g0 BWT is the day-8 incumbent being the uniform policy (finding 3), not transfer.

EvoHarnessBench's -34.7% BWT for agent-pool expansion under a fixed policy: here the pool expansion costs the fixed policies at the event (frozen hit -0.041, static -0.033 at g2) and they never recover (stationary J falls from 0.411 / 0.385 at g1 to 0.350 / 0.316 at g4), while the policy that moves loses nothing on the old stages.

(6) FWT is a property of the policy in service, not of anticipation. New capabilities arrive with zero logits, so the event-day policy is the previous stage's; the continual arm's FWT on g2–g4 ($+0.085$ / $+0.130$ / $+0.128$) exceeds the frozen arm's ($+0.046$ / $+0.057$ / $+0.032$) because it is a better policy, and the static router's FWT falls to -0.002 at g4 because its handbook prices unseen capabilities at a prior. Hits at the events are small for every arm except the price event, where the halved mid price lifts every mid-heavy policy by $+0.05$ to $+0.10$.

(7) The SkillOrchestra-style router is the weakest design-time baseline here, and the reason is diagnostic. The competence of an action is the mean outcome of episodes in which it was taken, which under per-step exploration is confounded by everything else in the episode: the handbook over-selects `verify@verify-1` (28% of decisions), an action the real run showed buys nothing. And λ_c is tuned by episode-level SN-IPS of a deterministic router against $\epsilon = 0.1$ traffic, whose ESS is 1–8 of 96, so the choice of λ_c is noise and the re-learned router's λ_c swings between 0 and 20 across stages. Re-learning per stage recovers some of the agent-pool loss ($+0.043$ at g4) at the price of three exploration days per stage, and still ends 0.07–0.11 below the oracle. This is a statement about applying the competence-minus-priced-cost rule to multi-step orchestration traces with a per-step router, not about SkillOrchestra's own handbook learning from richer per-skill traces on its own benchmarks.

Mild-schedule sensitivity. Figure 13 and Table 12: the ordering is unchanged with smaller gaps (continual 1.10, `never_verify` 0.39, frozen 1.97, static 2.90). In the mild run a learner exceeds the oracle at g1 (gap $+0.006$), a reminder that the oracle is a reference, not a proven optimum.

F.2 Verifier ablation: when does verification help?

Is "`never_verify` beats everything" an artefact of the measured verifier (uninformative: $P(\text{flag} \mid \text{wrong}) = 0.92$ against $P(\text{flag} \mid \text{correct}) = 0.96$) and of the 8-call cap that a verify-and-retry process fills? Four controlled variants

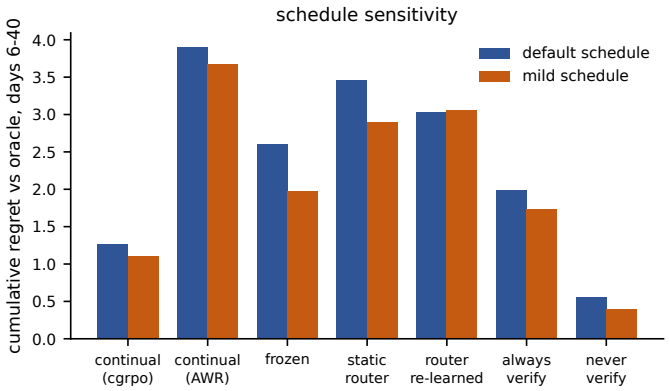


Figure 13: Cumulative regret under the default and the mild schedule. Same ordering, smaller gaps.

of the same 40-day stream, 5 seeds each (seeds 0–4; arms: continual cgrpo, `always_verify`, `never_verify`, per-stage oracle): A, the committed configuration (verify-1, cap 8), which also supplies the seed error bars above; B, the informative verify-2 for every arm from day 1 (per-call $P(\text{fail} \mid \text{good rows})$ $0.58 \to 0.114$, recall unchanged); C, B plus cap 12; D, C plus verifier calls exempt from the cap. Served numbers are means over all 40 days (the fixed modes do not learn); J uses the stage oracle's $\bar{\lambda}_g$.

Table 14: Verifier ablation: `always_verify` minus `never_verify`, served, mean $\pm$ SE over 5 seeds.

variant	Δ expected quality	ΔJ	Δ cost / task	cap hits (always)	retries / ep. (always)	verifier calls / ep. (always)
A: verify-1, cap 8 (committed)	-0.033 ± 0.001	-0.036 ± 0.001	+\$0.0036	95%	0.89	2.94
B: verify-2 for every arm	-0.004 ± 0.001	-0.007 ± 0.001	+\$0.0030	91%	0.64	2.90
C: B + cap 12	$+0.029 \pm 0.001$	$+0.011 \pm 0.001$	+\$0.0148	9%	0.76	3.65
D: C + verifier exempt from cap	$+0.028 \pm 0.001$	$+0.010 \pm 0.001$	+\$0.0148	0%	0.77	3.64

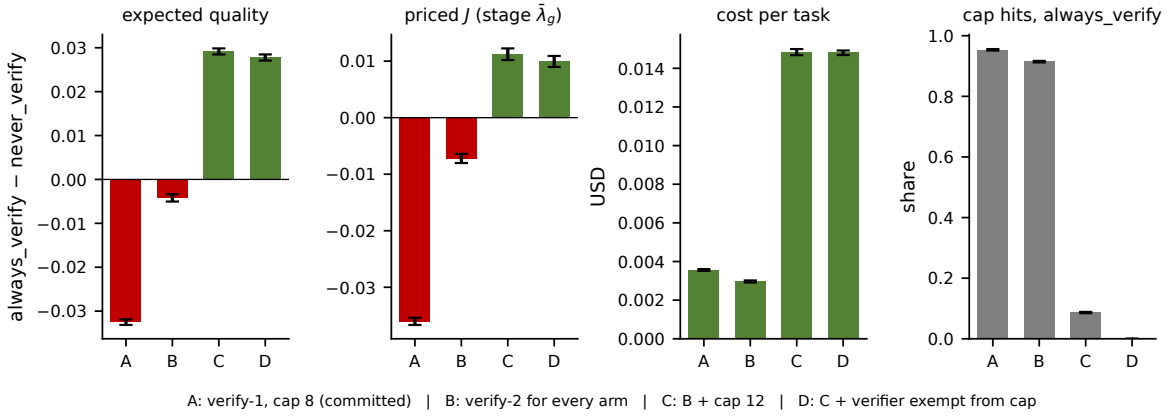


Figure 14: Verifier ablation (from the ablation summary file): `always_verify` minus `never_verify` in expected quality, priced J and cost, and `always_verify`'s cap-hit share, under the four variants; bars are means over 5 seeds with SE. The sign flips only when the verifier is informative *and* the retries do not come out of the plan's call budget.

Table 14 and Figure 14. Under the committed configuration verification *hurts*: `always_verify` is below `never_verify` by 0.033 ± 0.001 in expected quality and 0.036 ± 0.001 in J for +\$0.0036 per task, consistent with the real run's +1 task ($+0.019 \pm 0.067$ SE at $n = 50$). The informative verifier alone (B) closes the gap to -0.004 ± 0.001 but does not reverse it, because 91% of `always_verify` episodes still hit the cap and the plan is truncated. Raising the cap to 12 (C) flips the sign: $+0.029 \pm 0.001$ quality and $+0.011 \pm 0.001$ J for +\$0.0148 per task (+18%), with 3.65 verifier calls and 0.76 retries per episode and 9% cap hits (the stage-4 four-step plans). Exempting verifier calls from the cap (D) changes nothing beyond removing the last cap hits ($+0.028$ / $+0.010$): with one retry per sub-query a three-step plan already fits in 12 calls. Stated plainly: verification helps only when the verifier's fail is informative *and*

the retry does not come out of the plan's call budget, and then it is worth about $+0.03$ accuracy for $+18\%$ cost, a priced gain of $+0.01$ J at the stage prices. Under C and D the fixed `always_verify` costs \$0.098–0.105 per task, above the learner's B + tol = \$0.095, so it can exceed the budget-constrained oracle at a stage (g1 in C: 0.443 vs 0.434); the learner's verify share rises from 0.03–0.09 (A) to 0.10–0.13 (B–D) of its decisions and no further, and within budget it still ends the last stage above `never_verify` under every variant (g4: A $+0.031 \pm 0.007$, B $+0.017 \pm 0.011$, C $+0.016 \pm 0.007$, D $+0.013 \pm 0.010$). The whole-stream regret ordering under C and D is `always_verify` $0.30 <$ `never_verify` $0.60 <$ continual 1.05–1.20: with a verifier worth using and a cap that does not truncate, the right fixed mode is `always_verify`, it is over budget, and the budgeted learner is third, its cold start still the whole deficit. Two caveats. The informative verifier's episode-level $P(\text{flag} \mid \text{correct})$ in the simulator is 0.71, not the 0.30 its per-call parameter was derived from, because correct episodes still contain empty and wrong steps that are rightly flagged; the per-call parameter is what the ablation sets. And the effect size rests on the assumed link from a step's latent quality to the answer.

Table 15: The real log (130 MAS episodes): what a flagged step's retry did. Observation kind before the flag (rows) against after the retry (columns).

before → after	rows	empty	error	invalid	respond	total
rows	26	2	5	0	0	33
empty	5	7	0	1	0	13
error	5	1	0	0	0	6
invalid	0	0	0	0	1	1

The real log gives the per-step number beside the per-episode one (Table 15): 107 verifier fails, 53 followed by a retry of the same sub-query (the rest hit the cap or the one-retry limit). The retry changed the observation kind in 10 of 53 cases (19%), always from empty or error to rows: a flagged empty, error or invalid step became rows in 10 of 20 cases, so the retry does repair malformed or empty steps; but 33 of the 53 retries were of a step that had returned rows, and 26 of those returned rows again. When the retried step was verified again it passed 11 of 47 times (23%) against 42 of 114 (37%) for first attempts. Paired with `never_verify` on the same tasks, the 44 `always_verify` episodes that retried flipped 6 wrong→right and 5 right→wrong, with 16 unchanged right and 17 unchanged wrong (25% flipped, symmetric). Per step and per episode say the same thing: the measured verifier's flag is not information about correctness.

What this does not show. Effect sizes are the simulator's: the stage-2+ capabilities are assumed, as is the link from a worker step's latent quality to the answer. One schedule plus a mild one; the all-arms run is one seed, with 5-seed error bars for the continual, `always_verify`, `never_verify` and oracle arms from the ablation. The per-stage oracle is constrained GRPO on fresh exploration, not a proven optimum. Nothing here calls a model API; what a real-traffic run would require is in Appendix J.

G τ-bench and EvoHarnessBench: full tables

Section 4.4 states the findings; this appendix gives the setups and the per-suite tables. Both suites follow the counterfactual protocol of Section 4; their settings are in Table 21.

G.1 τ-bench

The agent is the tier model; the user simulator is fixed to the cheap tier for every arm, its cost logged and excluded from the agent cost. Quality is the benchmark's own reward, pass[1] per task. Episodes are long, 13–15 agent turns, each resending the policy document and tool schemas, so the cost column is three to four times the CRMArena-Pro column and the tier spread is wider. The budget for the constrained policies is the uniform behaviour's own mean agent cost (\$0.277 retail, \$0.294 airline), since no product budget exists for this suite.

On both domains the realised-log gate on the all-task constrained policy REVERTS on the quality bar (retail: DR 0.753 [0.613, 0.876] against a bar of 0.620; airline: 0.559 [0.346, 0.758] against 0.460) and 5% of redraws promote; with a binary reward and 49–60 tasks the DR quality interval is 0.30–0.40 wide. The decision that would capture the oracle's headroom is a mid-episode tier switch conditioned on the conversation so far, which this protocol does not log.

τ-bench retail, 60 tasks $\times$ 3 tiers	pass[1]	agent cost/task	held-out J ($\lambda = 2$) [fold SD]
fixed cheap	0.583	\$0.117	0.350 [0.135]
fixed mid	0.683	\$0.229	0.226 [0.154]
fixed frontier	0.717	\$0.487	-0.257 [0.176]
uniform behaviour	0.661	\$0.277	0.107 [0.146]
AWR+IPS linear, served	0.626	\$0.186	0.255 [0.135]
per-category rule / constrained ($B = \$0.28$), greedy	0.583 / 0.683	\$0.117 / \$0.229	= fixed cheap / = fixed mid
per-task oracle (max reward, cheapest among ties)	0.867	\$0.171	0.524 [0.080]

Table 16: τ-bench retail: fixed tiers, the oracle, and first-step policies (5-fold CV over task ids). 52 of 60 tasks are solved by at least one tier and 26 by all three. The first-step feature vector is the domain one-hot only, so every learned first-step policy collapses to a fixed arm.

τ-bench airline, 49 tasks $\times$ 3 tiers	pass[1]	agent cost/task	held-out J ($\lambda = 2$) [fold SD]
fixed cheap	0.449	\$0.099	0.251 [0.176]
fixed mid	0.551	\$0.261	0.027 [0.153]
fixed frontier	0.531	\$0.523	-0.516 [0.121]
uniform behaviour	0.509	\$0.294	-0.079 [0.121]
AWR+IPS linear, served	0.490	\$0.203	0.084 [0.134]
per-category rule / constrained ($B = \$0.29$), greedy	0.449 / 0.551	\$0.099 / \$0.262	= fixed cheap / = fixed mid
per-task oracle (max reward, cheapest among ties)	0.735	\$0.159	0.428 [0.064]

Table 17: τ-bench airline, collected to 49 of its 50 sampled tasks (the ledger cap bound before the last). 36 of 49 tasks are solved by at least one tier and 14 by all three; the mid tier beats the frontier tier at half its cost.

G.2 EvoHarnessBench, tools axis

The service exposes a harness that grows its tool set across six stages, $H_1 \subset H_2 \subset \cdots$; each task is run under the tool allowlist of its own stage, on all three tiers, with prompt caching on the fixed prefix (so the cost column is at cached prices and is not comparable to the other suites'). Score is the service's partial-credit grade, the fraction of hidden verifiers passed; pass requires all of them. Tasks were sampled by a seeded round-robin over (domain, split, cohort) groups covering both the held-out and the adaptation split, 50 per stage at stages 1, 2 and 4; stages 3, 5 and 6 were not collected.

stage		cheap	mid	frontier	per-task oracle	oracle picks c/m/f
	score	0.491	**0.702**	0.619	0.753	
H_1 (50)	pass	0.24	0.42	0.40	0.52	26 / 20 / 4
	cost	\$0.062	\$0.146	\$0.298	\$0.136	
	score	0.370	**0.699**	0.610	0.741	
H_2 (50)	pass	0.16	0.40	0.30	0.48	21 / 25 / 4
	cost	\$0.044	\$0.114	\$0.227	\$0.106	
	score	0.403	**0.643**	0.631	0.738	
H_4 (50)	pass	0.14	0.28	0.30	0.38	18 / 28 / 4
	cost	\$0.056	\$0.147	\$0.262	\$0.138	

Table 18: EvoHarnessBench tools axis, 150 tasks with all three tiers (held-out and adaptation splits pooled; 34–36 held-out per stage). Score is the mean verifier pass rate; cost is per task at cached prices. The oracle picks the cheapest tier among those with the best score.

At every stage the mid tier beats the frontier tier on score at about half its cost, and the oracle beats every single tier at or below mid-tier cost, choosing the frontier tier for only 4 tasks in 50. The ordering is the same at each of the three harness sizes, so growing the tool set did not change which tier is worth buying. Two things this collection does not contain, stated so that they are not read into it: every task ran under its own stage's harness, so all held-out tasks feed the benchmark's forward-transfer group and none of its backward-transfer cells (earlier-stage tasks re-run under later allowlists); and no controller was trained on these logs, since the stage-wise logs are of a size at which Appendix B.1 found first-step policies memorising. What the suite adds is the tier-level replication under a growing harness, on a

scorer that is not ours.

H Three follow-up experiments: delegation credit, meta-learned initialisation, and a skill handbook

Each follow-up changes one thing in the continual study or the first collection and is run over five seeds; each has the control that its result needed.

RAO-style delegation credit (negative). We added the node reward of Gandhi et al. [9] to the continual orchestrator's trainer: for a `delegate` decision, the per-decision reward is the episode's priced outcome plus λ_d times the mean success of the sub-call it spawned, with a leave-one-out baseline anchored on the root reward. Over five seeds on the committed simulator and schedule, the delegation share rose at every stage (from 0.17/0.44/0.50/0.51/0.55 across the five stages without credit to 0.26/0.47/0.57/0.62/0.62 with λ_d=0.5) and served regret over days 6–40 rose from 0.94 ± 0.27 to 1.74 ± 0.18; stationary priced J fell by 0.02–0.04 at every stage after the baseline. Because the credit mode also changes the baseline, we ran the same mode with λ_d=0: regret 0.99 ± 0.24 and unchanged shares. The credit, not the baseline, buys over-delegation. The learner without credit already delegated to new specialists from the stage they appeared; a bonus outside the baseline turns that into a habit.

Meta-learned controller initialisation (positive, caveated). The learner's deficit against the fixed policy was its cold start. Following the meta-learned initialisation of Tandon et al. [36], we learned the controller's initial weights by a first-order meta-loop over the simulator's stage definitions g_0–g_3 (never g_4, never a held-out set; audited per run), then ran the continual arm from that initialisation. Over five seeds, served regret over days 6–40 fell from 1.06 ± 0.21 to 0.47 ± 0.08 (paired -0.59 ± 0.15) and over days 17–40 from 0.76 ± 0.18 to 0.43 ± 0.07; the learner now has lower regret than `never_verify` (0.74 ± 0.08) on the full stream. The gain comes from what is served before and between promotions, not from earlier promotion (the first-promotion day is unchanged at 5.2 in this arm set), and there is no gain on g_4, the one stage the meta-initialisation never saw. The caveat is structural: the initialisation was trained on the simulator's own stage definitions; on real traffic it would have to be learned from past harness generations, which is the design-time loop of Figure 2.

Skill handbook for the cheap tier (null, with the control that showed it). The cheap tier scores 0.444 on the first 50 real tasks because it fails schema reasoning, while the mid and frontier tiers solve many of the same tasks. We distilled per-category procedural notes from those successful SOQL episodes with the mid-tier model as writer (leave-one-task-out, so no task saw notes from its own solution; 49 writer calls) and served the cheap tier with the notes in its prompt. Quality rose to 0.581 ($+0.137 \pm 0.070$ paired). But the 14 tasks that received no notes improved by the same amount as the 36 that did, so we re-ran the cheap tier with no notes: 0.561, a difference from the skilled arm of $+0.020 \pm 0.067$. The handbook did nothing measurable; the original 0.444 was a low draw. What survives is the size of that draw: the same tier on the same 50 tasks moved by $+0.117 \pm 0.062$ between two runs a day apart, which is as large as the gap between tiers in Appendix B.1, and a second, independent reason the gate of Appendix B.2 refused to certify a margin of that size.

I Advantage-weighted regression with inverse propensities

When no replay exists, so that a logged state cannot be re-rolled, the trainer is advantage-weighted regression [27] on logged steps with return $R_i = q_i - \lambda\, C_i^{\rightarrow}$ (cost-to-go from the step), a per-bucket value baseline $\hat{V}$, and weights corrected by the logged propensity and self-normalised over the batch:

$$A_i = R_i - \hat{V}(b(x_i)), \qquad w_i = \frac{\exp(A_i/\beta)/\mu_i}{\sum_j \exp(A_j/\beta)/\mu_j}, \qquad \max_W \sum_i w_i \log \pi(a_i \mid x_i). \tag{12}$$

Here λ is a fixed price. Logged counterfactual arms, when present, are added as extra records; offline RL can only reweight what the log contains. The AWR arm appears in the first CRMArena-Pro collection (Appendix B.1), the 300-task results (Section 4.1), the τ-bench tables (Appendix G) and the continual study, where it is the cautionary row: with the largest distance from the behaviour policy (effective sample size 25% against 54% for the replay trainer) it is the one arm the gate promoted wrongly.

J What a real-traffic continuation of the continual study would need

The same loop on the real orchestrator: days of 20 B2B tasks, the learner trained with AWR+IPS on the day's log (no replay exists on the real harness) and gated with the per-decision estimators, held-out evaluation by re-serving a fixed 20-task set per stage. Stages that can be induced for real: a repricing (free, with costs recomputed from logged tokens); a better verifier (swap its prompt or tier); a new worker or answer backbone (any priced model); a worker swap by tier; a schema-lookup tool (one describe call) or a local aggregation tool; skills as steering text (tokens only). Such a run needs about 500 served episodes per gate for the cost criterion to bind, as the simulator study found, and a quality bar set from the incumbent's own interval rather than its point estimate. The gate's quality criterion will not certify improvements at that size; it will only reject regressions, which is the use the discussion recommends.

K Extended related work

Cost-aware routing. FrugalGPT [5] cascades models by a learned score to cut cost; RouteLLM [26] learns a router between a strong and a weak model from preference data; xRouter [29] trains a cost-aware orchestration system with reinforcement learning. These choose a model per query at the first turn. This paper prices every decision point inside a frozen harness, learns the price as a dual variable against a budget, and gates deployment off-policy. SkillOrchestra [41] routes each query to a typed agent in a pool by a learned competence profile with an explicit cost penalty, learned once from traces and then served statically; the continual study implements its literal routing rule as a baseline and finds, in finding (7) of Appendix F, why a competence learned from per-step exploration traces of a multi-step orchestrator is confounded.

Self-evolving agent systems and the deployment seam. A position paper from the AReaL group [42] argues that enterprise self-evolving agents are blocked not by RL algorithms but by systems: a step-level episode-log protocol, a data proxy that turns production workloads into governed learning material, and a unified *evolution control plane* that selects, from log statistics, drift indicators and cost, among memory updates, skill updates, harness edits, weight updates, rollback or no-op, each behind replay-first evaluation, regression tests and canary rollout. This paper instantiates part of that architecture: the gate is a measured, interval-valued form of their replay-first check; priced backward and forward transfer are the drift indicators they call for; and the continual orchestrator is a per-request learner inside the harness surface they name, with the harness-edit surface left to the slow loop. Their prototype realises the weight-update path by redirecting deployed agents' inference calls through managed workers, the same seam that Agent Lightning [24] uses to make training harness-agnostic through a proxy that emulates the inference APIs. Placing the controller in that proxy is the deployment path we would take, since tier choice, verification and stopping are decisions at the API boundary, while re-planning and delegation still need a hook inside the orchestrator. Production precedent for the cadence exists: online RL after deployment has shipped with checkpoints refreshed from user interaction every few hours.

Recursive delegation and priced effort. Recursive Agent Optimization [9] trains agents that spawn instances of themselves through a `launch_subagent` call, with a local node reward equal to the node's own success plus λ_d times the *mean* success of its children (averaging rather than counting, which removes the incentive to spawn), a leave-one-out baseline anchored on the root reward, and depth-level inverse-frequency weighting so that deep sub-tasks do not dominate the update. Four points carry over. That node reward is the principled process reward for the delegate action of Appendix D, where our process rewards were definitional; the follow-up of Appendix H adopts its form and finds that it buys over-delegation under a price. Depth weighting is the recursive counterpart of the per-decision estimators used here. Delegation in RAO is unpriced, with depth and step caps standing in for a budget, so adding $\lambda \cdot \text{cost}$ and $\mu \cdot \text{latency}$ to the node reward is the one-line extension that turns the caps into a learned price. And RAO's generalisation from medium to hard tasks concerns the learned decomposition skill, not drift of the harness around it, which is what the transfer metrics here measure. On the shaping of the price itself, the Composer 2 report describes a nonlinear effort penalty over tokens, tool calls and turns whose slope is steeper on easy tasks than on long-horizon ones; the objective here uses a single linear λ, and a difficulty-conditioned price, λ per task category or a concave cost curve, is a direct extension that the logs here are too small to fit.

Contextual bandits in production. The Decision Service [1] and Horizon [10] established the logging discipline (propensities with exploration), the offline evaluation [4, 22] and the promotion workflow that the gate follows; the estimators are those of Dudík et al. [8], Jiang and Li [14], Swaminathan and Joachims [35], Thomas and Brunskill [38], with cross-fitting after Chernozhukov et al. [7].

Test-time compute and self-refinement. Socratic Self-Refine [32] decomposes an answer into verifiable sub-questions and rewrites where confidence is low; the uncertainty signals (verbalised confidence, sample agreement) follow Bakman et al. [3]. Appendix E treats how much of it to buy per request as a priced decision and finds that, on a CRM benchmark, a better tier up front dominates.

Process verification and reward modelling for multi-agent systems. MAS-ProVe [40] verifies multi-agent executions at the level of the individual agent call or of one iteration, online, by sampling three candidates and keeping the one the verifier ranks highest, across six frameworks and five benchmarks; it finds the gains inconsistent and high-variance, generative judges ahead of reward models, and step-only context as good as the full history. Its unit of verification is the decision point studied here, and its finding is one this paper reproduces on a priced log: the verification itself is a purchased action, three calls for one, and whether it pays is a per-decision question. Reward modelling for orchestration [39] scores an orchestration plan before any sub-agent runs, with a Bradley–Terry model trained on preference pairs mined from execution logs, so that best-of-N selection and orchestrator training avoid full rollouts; it is a reward model, not an estimator, and the natural combination with the gate is as the direct model inside DR. MAS-Orchestra [15] trains a holistic orchestrator with an outcome-only reward and asks when a multi-agent system beats a compute-matched single agent. ToolOrchestra [34] trains a stepwise orchestrator over tools and models with a composite reward that includes an efficiency term, the closest prior use of a priced orchestration reward, though unpriced at the per-decision level and without an off-policy gate. Appendix D measures the first three on one log and finds the verifier uninformative, the process features without out-of-fold signal, and the single agent ahead.

Harness evolution and continual-learning metrics. The design-time line [2, 6, 12, 21, 46, 47] and EvoHarnessBench [16] evolve or grow the harness; the BWT and FWT definitions are those of Lopez-Paz and Ranzato [23], applied here to a priced objective at the orchestrator level. Meta-Harness in particular proposes a new harness by reading the full execution history of the previous ones; it is the natural producer of H_{t+1} in the design-time loop of Figure 2, and the gate here is what would decide whether a controller trained on H_t still serves under it.

Offline and constrained RL. CQL [19] and IQL [18] are the standard pessimistic offline learners; AWR [27] is the simpler weighted-regression trainer used here with inverse propensities; GRPO [31] supplies the group-relative advantage and DAPO [45] the dynamic-sampling and clipping variants used in the platform controls; the learned price follows RCPO [37] and the PID-Lagrangian [33], with iterate averaging after Nedić and Ozdaglar [25].

L Limitations and threats to validity, in full

- **Sample sizes.** The real-harness evidence is 300 B2B and 150 B2C tasks on CRMArena-Pro, 60 and 49 tasks on the two τ-bench domains, and 50 tasks at each of three EvoHarnessBench stages, every task on every tier. The held-out numbers of Appendix B.1 are means over 10 tasks per fold, where one task is 0.1 of quality and the across-fold SDs are as large as the differences between policies; the 300-task section is where the orderings become stable, and the gate's intervals are still five times the budget margin there. The multi-agent run has 130 episodes on the first 50 tasks; the self-refinement run 41 tasks. The negative results for the ORM, the refinement gate and the first-step policies on τ-bench are statements about these sample sizes and observables, not about the approaches.
- **Seeds.** The simulator pipeline reports 5 seeds; the CRMArena folds one split, and each GPU run ran once; the continual study's all-arms run is one seed per schedule, and only the verifier ablation and the follow-ups (continual, the two fixed modes and the oracle) report 5-seed error bars. The router and AWR arms carry no seed variance.
- **Simulators calibrated to the project's own logs.** The simulator of Appendix A is hand-written. The continual simulator's process statistics are measured on 130 episodes, but the link from a step's latent quality to the answer and every capability that arrives after stage 0 are assumed (Appendix Q); the transfer effect sizes are the simulator's, not CRMArena's. The calibration check reproduces the fixed modes' cost and call counts but lands accuracy at $0.446/0.459$ against $0.480/0.460$, because the simulated verifier is uninformative, as the measured one is.
- **Assumed stage parameters.** $P(\text{good} \mid \text{rows}) = 0.60/0.75/0.85$ by tier; coverage-to-answer logit slope 1.5; task-level logit noise sd 0.4; `verify-2` at $P(\text{flag} \mid \text{correct}) = 0.30$ with the same recall; `data-4` at rows 0.85 / good 0.80; `answer-4` mid-grade on 8 categories; `schema_lookup` removes error mass from the next step; `aggregate_calc` adds $+0.34$ coverage on aggregation categories; the three skills add $+0.15$ $P(\text{good} \mid \text{rows})$ on their categories or re-plan into 3–4 sub-queries; planner v2 plan length $(0.0, 0.1, 0.4, 0.5)$. Tools and skills are priced as a verifier-sized cheap call (711/62 tokens).
- **The SkillOrchestra baseline is the literal rule, not their system.** Competence minus λ_c times mean logged cost, over task categories as skills, learned from per-step exploration traces of a multi-step orchestrator. Their handbook

learning from richer per-skill traces on their own benchmarks is not reproduced.

- **The oracle is a reference.** Constrained GRPO on fresh exploration with more iterations, not a proven optimum; regret can be slightly negative.
- **Gate design.** The simulator gate fits its direct model in-sample; the bandit gate cross-fits by task; the continual gate uses the sequential DR with cumulative ratios clipped at 50, whose bias on far-from-behaviour candidates is what finding (4) of Appendix F exposes.
- **The τ-bench first-step policies.** With the domain one-hot as the only step-0 feature, the learned first-step policies cannot differ from a fixed arm; the suite bounds the headroom and says what to log next, and nothing more.
- **EvoHarnessBench pricing and coverage.** Its cost column is at cached prices; stages 3, 5 and 6 and the backward-transfer cells were not collected; no controller was trained on its logs.
- **Run-to-run variation of a tier.** The same tier on the same 50 tasks moved by $+0.117 \pm 0.062$ between two runs a day apart (Appendix H), which bounds what any held-out comparison at that scale can certify.
- **Data terms.** CRMArena-Pro is CC BY-NC 4.0 (research use), fetched at setup and not copied; the committed traces are derived research artefacts under the same terms (Appendix T).

M Experimental details

One table per experiment. Values are taken from the run metadata files, the command-line defaults of the scripts that produced each result, and `results/llm/backends/COMPARISON.md`; a setting that none of these records is marked "not recorded". Costs in the CRMArena-Pro rows are token counts times the list prices given in Table 20; simulator costs are in simulator units.

Table 19: Simulator pipeline (Appendix A; `scripts/run_v0.sh`, `scripts/run_sim_seeds.sh`).

setting	value
harness	simulator v_1; drift event v_2 plus reprice(mid, 1.2); at most 4 steps; 12 actions
policy class	softmax-linear, Eq. (4); 48 features; logit clip $L = 3$; served exploration $\epsilon = 0.05$
trainer	constrained GRPO on replay (`pace.train_offline --method cgrpo`; NumPy, no RL library)
group size / batch	$G = 8$ continuations per state; 32 logged states per iteration
iterations	300 on v_1; 300 for the v_2 retrain, warm-started from the v_1 policy
learning rate	1.5 (code default)
ratio clip; KL	none; none (on-policy group-relative gradient, Eq. (5))
reward	realised 0/1 outcome minus λC
λ learning	dual step $\eta = 0.1$, cost EMA $\rho = 0.9$, damping $k_p = 2.0$ (Eq. (3)); inferred from the logged λ and cost EMA of the first iterations, not stored as settings); served $\bar{\lambda}$ and policy averaged over 30 snapshots from the second half
budget	$B = 0.30$
behaviour data	v_1: 3,000 episodes, uniform behaviour; v_2: 6,000 episodes served by the frozen v_1 policy with $\epsilon = 0.3$; propensities logged with exploration
gate	IPS, SNIPS, DR (in-sample bucketed direct model); cumulative ratio clip 50; 200 bootstrap resamples, 90% percentile intervals; $\epsilon_q = 0.03$, tol 0.02; v_2 bar from the v_1 promotion
ground truth	Monte Carlo over 20,000 fresh tasks, expected success probability
seeds	replicate $s \in \{0, \ldots, 4\}$: collect / train / collect / train seeds $10s{+}1, \ldots, 10s{+}4$; gate bootstrap and ground-truth seed s ($s = 0$ is `run_v0.sh`)
hardware	CPU

Table 20: CRMArena-Pro collections and the offline analyses on them (Appendices B.1, B.2, D, E, 4.1).

setting	value
benchmark	CRMArena-Pro, b2b and b2c orgs, evaluator in `aided` mode (exact match, F1 for fuzzy categories, privacy judge)
tiers and list prices	cheap `claude-haiku-4-5`, mid `claude-sonnet-5-5`, frontier `claude-opus-5-5`; USD per million input / output tokens: 1.00 / 5.00, 2.00 / 10.00, 4.00 / 20.00; no prompt caching
counterfactual collection	b2b: 50 tasks (2–3 per category, all 22), extended on the same seeded, category-stratified sample to 150 and then 300; b2c: 150 tasks on the same protocol; every task on all three tiers; fixed continuation on the chosen tier; seed 0
propensity	$1/3$ at step 0 (uniform over tiers), 1 after
episode limits	12 agent turns (`ANSWER` forced at the last); 2,048 output tokens per agent turn, 1,024 for the forced answer; observations truncated to 20,000 characters; reasoning effort `low` for mid and frontier; no sampling parameters sent
offline learning on the log	AWR+IPS at $\lambda \in \{2, 5, 10\}$ per USD; constrained GRPO on the lookup table at $B = \$0.10$ with learned λ; 300 AWR epochs and 300 constrained-GRPO iterations (code default); 5-fold category-stratified CV over task ids, seed 0
gate on the realised log	bandit DR with cross-fitted direct model; 200 bootstrap resamples; 200 redraws of the one-arm log; $\epsilon_q = 0.03$, tol \$0.005
multi-agent run	same 50 tasks; planner cheap (600 output tokens, 1–3 sub-queries), data worker mid (one agent turn per sub-query), verifier cheap (200 output tokens, output shown truncated to 3,000 characters), answer mid; at most 8 role calls, 1 re-plan, 1 retry per sub-query; modes `always_verify` (50 episodes), `never_verify` (50), uniform exploration with logged propensities (30)
orchestration reward model	logistic regression and a 16-unit MLP; out-of-fold over tasks
self-refinement run	initial answers: the cheap-tier episodes of the counterfactual log; decompose on cheap (600 output tokens, at most 3 claims); $k = 2$ re-solve samples per claim, at most 3 turns each; refine (512 output tokens) on cheap and on mid; 42 tasks probed, 41 complete; controller: softmax-linear over 36 features, exact policy gradient on the logged tree, $B = \$0.10$ or $\lambda \in \{2, 5\}$, 5-fold CV over tasks
hardware	CPU client; models served by the provider's API

Table 21: The two further suites (Appendix G; `pace.experiments.taubench`, `pace.experiments.evoharness`).

setting	value
τ-bench	retail domain, 60 tasks, and airline domain, 49 of 50 sampled tasks (the ledger cap bound before the last); seeded shuffle of the task ids (seed 0); agent on the tier model, user simulator fixed to the cheap tier for every arm with its cost logged separately; reward the benchmark's own pass[1] from its environment checker; at most 30 agent turns, 2,048 output tokens per turn, 512 for the user turn; reasoning effort `low`; same tiers and list prices as Table 20, no prompt caching
τ-bench offline analysis	as Table 20: AWR+IPS at $\lambda \in \{2, 5, 10\}$, constrained GRPO on the lookup table at B = the uniform behaviour's cost (\$0.277), 5-fold CV over task ids; first-step features are the domain one-hot
EvoHarnessBench	hosted service, tools axis, stages 1, 2 and 4, 50 tasks each from a seeded round-robin over (domain, split, cohort) groups covering both the held-out and the adaptation split; each task under its own stage's tool allowlist on all three tiers; score the service's hidden-verifier pass rate, pass = all verifiers; at most 20 agent turns, 2,048 output tokens; reasoning effort `low`
EvoHarnessBench pricing	prompt caching on the system prompt (domain policy, action protocol, stage tool schemas) and the last message; cache writes priced at $1.25\times$ and reads at $0.1\times$ the input price; the cost column is therefore at cached prices and is reported separately from the other suites
hardware	CPU client; models served by the provider's API

Table 22: LLM controller, GRPO with LoRA (Appendix C; `results/llm/run_meta_phase1*.json`). The CRMArena-Pro run `crm_phase2` uses the `phase1c` recipe in lookup mode on fold 0.

setting	`phase1`	`phase1b`	`phase1c`
model	Qwen3-1.7B, bf16, LoRA $r = 16$, $\alpha = 32$, dropout 0.05 on q, k, v, o, gate, up, down projections		
library	TRL 1.15.0 (loss type `dapo`, group-scaled rewards), Transformers 5.19.0, PEFT 0.21.2, PyTorch 2.14.1		
prompts × group size	8×8	8×8	4×16
completions per step; micro-batch	64; 16	64; 16	64; 16
steps per generation	4	4	4
max completion length	16 tokens	16 tokens	16 tokens
learning rate	10^{-5}	10^{-5}	10^{-5} (SFT 2×10^{-4})
clip (low / high)	0.2 / 0.2 (library default)	0.2 / 0.2 (library default)	0.2 / 0.2 (library default)
KL coefficient; reference	0.02; base model	0.05; base model	0.05; SFT model (adapter merged)
entropy coefficient	0.0 (code default)	0.01	0.01
temperature	1.0 (code default)	1.0	1.2, raised to 1.5 at step 20
GRPO steps	300	300	200, after 150 SFT steps
SFT data	–	–	oracle labels on 2,000 tasks at shown prices $\{0.0, 0.1, \ldots, 2.0\}$; 1,000 held-out prompts
λ: dual step / EMA / k_p	0.1 / 0.9 / 2.0	0.02 / 0.95 / 0.5	0.02 / 0.95 / 0.5, $\lambda_0 = 0.5$, shown in prompt
budget	$B = 0.30$	$B = 0.30$	$B = 0.30$
training tasks	2,000 seeded simulator tasks (seed 0), legal-action order shuffled per task		
evaluation set	2,000 fresh simulator tasks, seed 777, greedy decoding		
seeds; hardware	one run each, seed 0; one NVIDIA A100-SXM4-40GB		

Table 23: Platform and recipe controls on the phase-1b setup (Appendix C, platform controls; `results/llm/backends/COMPARISON.md`). Shared by all arms: Qwen3-1.7B with LoRA $r = 16$, $\alpha = 32$; temperature 1.0; KL coefficient 0.05 against the base model (k3 estimator in the loss for verl and NeMo-RL); entropy coefficient 0.01; learning rate 10^{-5}; at most 16 completion tokens; price fixed at $\lambda = 0.44$ in the prompt and the reward; the phase-1c task pool (2,000 tasks, seed 0); one A100-40GB; one run each. Evaluation for every adapter: one evaluator (HF `generate`), greedy and sampled at $T = 1$, on the 2,000 seed-777 tasks with the $\lambda = 0.44$ prompt.

setting	TRL, fixed price	TRL + dynamic sampling	verl, DAPO recipe	NeMo-RL, DAPO recipe
version	TRL 1.15 (phase-1b stack)	TRL 1.15, HF `generate`	verl 0.9.1, vLLM 0.24	NeMo-RL NGC v0.7.0, vLLM 0.20
prompts × group size	4×16 (code default)	4×16	4×16	4×16
clip (low / high)	symmetric; 0.2 / 0.2 (code default)	0.2 / 0.28	0.2 / 0.28	0.2 / 0.28
dynamic sampling	no	yes (up to 8 generation rounds)	yes (`filter_groups`)	yes
loss aggregation	token mean over the accumulated batch, `dapo` (library default)	token mean over the accumulated batch, `dapo` (library default)	token mean	token level
steps completed	150	186 (adapter at step 150 evaluated)	117 (checkpoint at step 100 evaluated)	25 (training curves only, not evaluated)

Table 24: Continual orchestrator (Appendix F; `pace.experiments.mas_continual`). Simulator parameters are in Appendix Q.

setting	value
stream	40 days $\times$ 96 seeded tasks per day, the same tasks for every arm; warm-up 5 days
stage schedule	g1 reprice (day 9), g2 agents (day 17), g3 tools (day 25), g4 skills (day 33); default and mild schedules (Table 10)
held-out set	150 fixed tasks per stage, simulator expected quality, never seen by the gate
policy class	softmax-linear over 18 actions and 88 features; served $\epsilon = 0.1$; zero feature slots and logits for capabilities not yet present
trainer (continual)	constrained GRPO on replay from the last 5 days of logged states; 60 iterations per day, batch 16, group 6; warm start from itself; iterate averaging
trainer (AWR variant)	AWR+IPS, fixed $\lambda = 2$ per USD, 200 epochs
learning rate; ratio clip; KL	1.5 constrained GRPO, 2.0 AWR (code default); none; none
λ learning	learned against $B = \$0.090$ per task; dual step 20, cost EMA 0.9, damping $k_p = 40$, per USD of cost gap (code default)
gate	sequential DR, cumulative ratios clipped at 50; 200 bootstrap resamples; $\epsilon_q = 0.05$, tol \$0.005; log since the last harness event, at most 5 days; budget net of the policy-independent prefix
per-stage oracle	constrained GRPO on 200 fresh exploration episodes, 300 iterations, batch 24, group 8, no gate
SkillOrchestra-style router	handbook from warm-up exploration (days 1–4), λ_c chosen on day 5 by episode-level SN-IPS; re-learned variant: 3 days at $\epsilon = 0.3$ per stage, then refit
prices	list prices of Table 20, scaled by the stage's repricing
metrics	recovery tolerance 0.02; stationary window 4 days; regret over days 6–40
seeds; hardware	all arms seed 0 (default and mild); verifier ablation and the follow-ups seeds 0–4; CPU

Table 25: Simulator pipeline, per seed (`results/sim_seeds/RESULTS.md`). DR quality with its 90% CI; "cost hi" is the DR cost upper bound against $B + \text{tol} = 0.32$; truth is Monte Carlo over 20,000 tasks (for v_2, the adapted policy); recovery as served after the gate and for the adapted policy.

seed	$\bar{\lambda}$	gate v_1 DR quality	cost hi	decision	truth v_1 q / c	gate v_2 DR quality	cost hi	decision	truth adapted q / c	priced-BWT	recovery (served)	recovery (adapted)
0	0.343	0.851 [0.815, 0.887]	0.309	PROMOTE	0.880 / 0.303	0.873 [0.854, 0.889]	0.311	PROMOTE	0.871 / 0.312	-0.045	$+0.033$	$+0.033$
1	0.379	0.880 [0.848, 0.911]	0.310	PROMOTE	0.877 / 0.310	0.871 [0.851, 0.892]	0.292	PROMOTE	0.864 / 0.292	-0.049	$+0.042$	$+0.042$
2	0.278	0.901 [0.878, 0.921]	0.313	PROMOTE	0.877 / 0.309	0.879 [0.860, 0.895]	0.304	REVERT	0.869 / 0.304	-0.041	-0.000	$+0.034$
3	0.443	0.883 [0.845, 0.905]	0.313	PROMOTE	0.878 / 0.306	0.855 [0.825, 0.881]	0.290	REVERT	0.864 / 0.288	-0.057	$+0.001$	$+0.051$
4	0.346	0.903 [0.870, 0.925]	0.308	PROMOTE	0.877 / 0.302	0.867 [0.847, 0.887]	0.296	REVERT	0.866 / 0.296	-0.042	$+0.000$	$+0.033$

N A logged episode, step by step

One task from the B2B counterfactual log, `b2b-1755` (category `lead_routing`: which sales user a lead in a given state should be routed to), as the platform recorded it. Every step carries the action as an (op, tier) pair, the propensity the behaviour policy assigned to it, the realised cost and latency, the 43-dimensional feature vector and the legal action set. The mid-tier arm:

```
t=0  CONTINUE@mid   propensity 0.333   $0.0141   1.4 s
t=1  CONTINUE@mid   propensity 1.0     $0.0146   1.6 s
t=2  CONTINUE@mid   propensity 1.0     $0.0186   2.9 s
t=3  CONTINUE@mid   propensity 1.0     $0.0172   1.2 s
quality 1.0   total cost $0.0645   latency 7.1 s   end: agent_respond
```

The agent's logged tool calls at those steps: a query for the territory whose description matches the state; a query for the users assigned to that territory; a count of accepted quotes per owner among those users; the answer, one user id, which matched the reference exactly. The propensity is $1/3$ at $t = 0$ because the collection's behaviour policy drew the tier uniformly, and 1 afterwards because the continuation is fixed on the chosen tier (Appendix B.1); the controller's decision is the first step only, so later steps carry no weight in any estimator.

arm	steps	answer	quality	cost
cheap	7	a different user id	0.0	\$0.049
mid	4	correct	1.0	\$0.065
frontier	4	correct	1.0	\$0.128

Table 26: The same task on the three arms. The per-task oracle is the mid tier: best quality, cheapest among the correct arms.

What the controller does with it. At $t = 0$ the feature vector has two non-zero entries, the task category and the harness version. The constrained policy of Section 4.1 (trained on all 300 tasks, $\bar{\lambda} = 1.09$ per dollar) maps it through a 3×43 weight matrix with logit clip 3 to

$$\pi(\text{cheap} \mid x) = 0.054, \qquad \pi(\text{mid} \mid x) = 0.009, \qquad \pi(\text{frontier} \mid x) = 0.937.$$

It has learned that `lead_routing` tasks, on average over the log, are worth the frontier tier under its budget; on this task that spends twice what the mid tier needed. The gap between the learned policy and the per-task oracle in Table 1 is this gap: the policy conditions on the category, the oracle on the outcome.

What the gate does with it. On a realised one-arm log that drew the mid tier for this task, the candidate's importance weight is $\pi(\text{mid} \mid x)/(1/3) \approx 0.03$, so the episode barely enters the IPS term; a task whose log drew the frontier tier carries weight 2.8. The doubly-robust estimator (Appendix P) first predicts the task's quality and cost from its category with a direct model fitted on the other tasks, then adds the weighted residual. Over the 300 tasks this gives quality 0.607 $[0.525, 0.668]$ and cost $\$0.095$ $[0.087, 0.113]$; the quality lower bound clears $q_{\text{prev}} - \epsilon = 0.484$, the cost upper bound does not clear $B + \text{tol} = \$0.105$, and the gate REVERTS. The policy's exact cost on the full log is $\$0.098$, inside the budget by less than the tolerance, which is the case the interval cannot certify at this sample size (Appendix B.2).

O Prompts

The templates below are copied from the code. Text in braces, `{like_this}`, is filled at run time; everything else is literal. Where a prompt is assembled by a function rather than stored as a constant, the listing is the exact output of that function on a seeded example, with the variable fields named in the paragraph heading. The CRMArena-Pro agent's own system prompt (the benchmark's ReAct prompt with the rendered org schema and metadata, built by `system_prompt()` in `pace/harness/crmarena.py` from the vendored `crm_sandbox.agents.prompts`) is the benchmark's text under its licence and is not reproduced; the planner and the worker run inside that system prompt, and the verifier and the self-refinement roles do not.

Controller, system message (`pace/llm/env_text.py`, `SYSTEM`).

```
You are the routing controller of an agent harness. You pick which model tier runs the next operation. Reply with exactly one
    action tag and nothing else.
```

Controller, simulator user message (`sim_prompt`; rendered for sim task 0, seed 0, $B = 0.30$; difficulty proxy, state, prices, the shuffled legal list and the example action vary per task).

```
Task: answer one question. Observed difficulty: hard (a noisy estimate, right about 85% of the time).
State: step 0, evidence 0, verified no, cost so far 0.000.
Budget: average cost per task must stay at or below 0.30.
Price of one ANSWER call by tier: cheap 0.020, mid 0.100, frontier 0.400. RETRIEVE and VERIFY cost 0.5x (RETRIEVE adds a 0.005
     search fee), ESCALATE costs 1.5x.
Ops: ANSWER = answer now; RETRIEVE = fetch evidence; VERIFY = check with a judge; ESCALATE = answer now with extra compute.
After a non-final op the harness answers on the same tier.
Legal actions: mid ANSWER, cheap ESCALATE, frontier VERIFY, frontier RETRIEVE, mid ESCALATE, frontier ANSWER, cheap RETRIEVE,
    mid VERIFY, mid RETRIEVE, cheap ANSWER, frontier ESCALATE, cheap VERIFY.
Choose the action that maximizes expected answer quality within the budget.
Reply with exactly one tag, for example <action>frontier VERIFY</action>.
```

Controller, price-conditioned variant (phase 1c and the backend runs): the lines that differ, at $\lambda = 0.44$ (shown rounded to 0.1).

```
Price of cost: λ=0.4 (your reward is quality − λ·cost).
Choose the action that maximizes expected quality − λ·cost.
```

Controller, CRMArena-Pro lookup mode (`lookup_prompt`; $B = \$0.10$, typical episode costs from the 50-task log, legal list unshuffled; an optional `Query:` line follows the first line when the task text is included).

```
Task: a CRM agent task (org {org}, category {category}, scored by {reward_metric}).
Budget: average cost per task must stay at or below $0.100.
Typical cost of a full episode by tier: cheap $0.0680, mid $0.1196, frontier $0.1642.
CONTINUE = run the agent on that tier until it answers.
Legal actions: cheap CONTINUE, mid CONTINUE, frontier CONTINUE.
Choose the tier that maximizes expected answer quality within the budget.
Reply with exactly one tag of the form <action>TIER OP</action> naming one legal action.
```

Planner, user message (`pace/experiments/mas/roles.py`; system message: the benchmark's ReAct prompt).

```
Request: {query}

[MAS PLANNER] You are the planner of a CRM data team. Decompose the user's request into 1 to 3 sub-queries that a data worker
     will run one at a time against the Salesforce instance described above. Each sub-query has a `goal` (what it should
     establish) and a candidate `soql` (SOQL or SOSL text; leave it empty if the worker should write it). Order them so later
     sub-queries can use earlier results. Reply with JSON only:
{"subqueries": [{"goal": "...", "soql": "..."}], "rationale": "one sentence"}
```

Planner, re-plan user message.

```
Request: {query}

Evidence so far:
- {evidence item, at most 400 characters}
...

[MAS PLANNER] You are re-planning. The evidence gathered so far is listed below, with the verifier's findings. Produce the
     REMAINING sub-queries (1 to 3) needed to answer the request; do not repeat sub-queries whose results were verified. Same
     JSON format.
```

Worker, steering text appended to the agent's running ReAct context (the `Candidate SOQL` line only when the plan gave one).

```
[MAS WORKER] Sub-query {k} of {n}: {goal}
Candidate SOQL: {soql}
Run exactly one query now with <execute>...</execute> (adapt the candidate if needed).
```

Worker, steering text on a retry after a verifier fail.

```
[MAS WORKER] Sub-query {k} of {n}: {goal}
Candidate SOQL: {soql}
The verifier found a problem with your previous attempt: {reason}. Fix the query and run it again.
Run exactly one query now with <execute>...</execute> (adapt the candidate if needed).
```

Verifier, single user message (no system message).

```
Request: {query}
Sub-query goal: {goal}
Query run: {executed query}
Output: {Salesforce output, truncated to 3,000 characters}

[MAS VERIFIER] You verify one step of a CRM data worker. Given the user's request, the sub-query goal, the query that was run
     and the Salesforce output, decide whether this step's result is usable: well-formed, non-empty when the goal needs rows,
     and consistent with the goal (right object, right filter, right field). Reply with JSON only: {"pass": true|false, "
     reason": "one sentence"}
```

Answer: the harness's `ANSWER@tier` op appends this to the last user turn of the agent's context (`pace/harness/crmarena.py`, `FORCE_ANSWER`).

```
You must give your final answer now, without running more queries. Output only <respond> your final answer </respond>.
```

Self-refinement, decompose: system message (`pace/experiments/ssr/refine.py`).

```
You audit answers produced by a CRM assistant that queries a Salesforce instance with SOQL/SOSL. You do not run queries
     yourself. You decompose a proposed answer into verifiable sub-claims and estimate how likely each is to be true. Be
     calibrated: a confidence of 0.9 should be wrong about one time in ten.
```

Self-refinement, decompose: user message (a Python format string: single braces are fields, doubled braces are literal braces).

```
Task: {task_id}
Question asked of the assistant:
{query}

Queries the assistant ran before answering ({n_queries}):
{queries}

Proposed answer:
{answer}

Decompose the proposed answer into 1 to {max_claims} sub-claims that together imply the answer. For each sub-claim write one
     Socratic question that a worker can settle by running SOQL/SOSL on the same Salesforce instance, the value the sub-claim
     asserts (short, literal), and your confidence in [0, 1] that the sub-claim is true. Then give your confidence that the
     proposed answer as a whole is correct. If the proposed answer is a refusal or "None", make the single claim "the request
     should be refused / cannot be answered" with a question that checks whether the data is accessible.
Output JSON only, no prose:
{{"claims": [{{"claim": "...", "question": "...", "expected": "...", "confidence": 0.0}}], "answer_confidence": 0.0}}
```

Self-refinement, re-solve: suffix appended to each Socratic question run as a worker mini-episode.

```
Answer this sub-question only. Run the queries you need, then reply with <respond> the value </respond> and nothing else in
     the response.
```

Self-refinement, refine: system message.

```
You finalize answers produced by a CRM assistant that queries a Salesforce instance. You are given the question, the proposed
     answer, and the findings of independent workers who re-checked each sub-claim with their own queries. Keep the proposed
     answer when the findings support it; correct it when they contradict it; keep a refusal when the request concerns
     confidential data. Do not hedge, do not run queries, and output exactly one <respond> ... </respond> block with the final
      answer in the same format as the proposed answer.
```

Self-refinement, refine: user message (format string).

```
Task: {task_id}
Question:
{query}

Proposed answer:
{answer}

Sub-claims and worker findings:
{findings}

Give the final answer now. Output only <respond> final answer </respond>.
```

P Estimator derivations

Unbiasedness of per-decision IPS. For a logged episode drawn from μ, and any step t, $\mathbb{E}_\mu[\rho_{0:t}\, r_t] = \sum_{a_{0:t}} \prod_{k\le t} \mu(a_k \mid s_k) \prod_{k\le t} \frac{\pi(a_k|s_k)}{\mu(a_k|s_k)}\, r_t = \mathbb{E}_\pi[r_t]$, whenever $\mu(a \mid s) > 0$ wherever $\pi(a \mid s) > 0$; the ϵ-mixture of Eq. (4) guarantees the support condition for every served log. Summing over t gives Eq. (7). The clip at $\rho_{\max}$ trades this unbiasedness for variance.

The doubly-robust decomposition. Write $\delta_t = r_t + \hat{V}(s_{t+1}) - \hat{Q}(s_t, a_t)$, the temporal-difference residual of the direct model under the candidate. Eq. (8) is $\hat{V}(s_0) + \sum_t \rho_{0:t}\delta_t$. If $\hat{Q} = Q^\pi$ then $\mathbb{E}_\mu[\rho_{0:t}\delta_t \mid s_t, a_t] = 0$ step by step and the estimate equals $\hat{V}(s_0) = V^\pi(s_0)$ regardless of the ratios; if instead the ratios are exact (no clip) then telescoping the $\hat{V}$ and $\hat{Q}$ terms recovers IPS regardless of $\hat{Q}$. Either model being right is enough [14]. With a clip, the error is $\sum_t(\rho_{0:t} - \tilde{\rho}_{0:t})\delta_t$, which is small when the direct model's residuals are small: the clip biases DR only through the direct model, as stated in Section 3.1.

Bandit DR with cross-fitting. In Eq. (9), the direct-model term $\sum_a \pi(a \mid x_i)\hat{Q}^{(-i)}(x_i, a)$ is evaluated on a model that did not see task i's reward, so $\mathbb{E}[w_i(r_i - \hat{Q}^{(-i)}(x_i, a_i)) \mid x_i]$ is the reward model's error under π, not a term correlated with r_i by construction. With an in-sample model, $\hat{Q}(x_i, a_i)$ contains r_i itself; with buckets of a few samples the correction term shrinks toward zero and the estimate is pulled toward the direct model. Appendix B.2 reports both choices empirically (coverage 99%/92% cross-fitted against 96%/94% in-sample with a -0.006 quality bias).

Why every arm logged makes the estimators exact. With three arms per task each at propensity $1/3$, the weights over a task's three episodes are $3\pi(a \mid x)$ and sum to $3\sum_a \pi(a \mid x) = 3$; IPS over the 150 episodes is therefore $\frac{1}{50}\sum_i \sum_a \pi(a \mid x_i) r(x_i, a)$, the exact value, and SNIPS and DR coincide with it. The remaining interval is task-sampling uncertainty.

The ϵ-mixture score. From $\pi_\epsilon = (1-\epsilon)\pi + \epsilon/|\mathcal{A}|$, $\nabla\pi_\epsilon = (1-\epsilon)\nabla\pi = (1-\epsilon)\pi\nabla\log\pi$, so $\nabla\log\pi_\epsilon = \frac{(1-\epsilon)\pi}{\pi_\epsilon}\nabla\log\pi$, Eq. (5). Using the bare score would weight exploration draws as if the policy had chosen them.

Effective sample size. For self-normalised weights, the variance of $\sum_i w_i r_i / \sum_i w_i$ behaves like that of a simple mean over $\text{ESS} = (\sum_i w_i)^2 / \sum_i w_i^2$ draws [17]; the report flags any estimate with $\text{ESS} < 0.1\, n$.

Q Continual simulator: calibrated and assumed parameters

Table 27: Every parameter of the continual simulator with its status (measured from the committed logs, with the count, or assumed).

parameter	value	status	source / note
initial plan length $P(1, 2, 3)$	0.031 / 0.169 / 0.800	measured	130 planner calls (4 / 22 / 104)
re-plan length	1 sub-query	measured	15 re-plans, all emitted 1
data-2 (mid) step kind rows/empty/error/invalid/respond	0.761 / 0.151 / 0.057 / 0.027 / 0.003	measured	331 worker calls
data-1 (cheap) step kind	0.36 / 0.09 / 0.09 / 0.00 / 0.45	measured, thin	11 worker calls
data-3 (frontier) step kind	0.80 / 0.10 / 0.10 / 0.00 / 0.00	measured, thin	10 worker calls
retried step: same kind distribution as a first attempt	rows 0.68 vs 0.76	measured-consistent	53 retries vs 299 first attempts
tokens planner / worker / verifier / answer (in, out)	5,029/279; 8,369/158; 711/62; 12,250/91	measured	call-log means per role
worker input tokens by position	$6833 + 1748k$	measured	positions 0/1/2: 6,833 / 8,571 / 10,330
cheap / frontier tokens per role	planner 6,996/159; 6,902/311; worker 6,647/115; 9,598/196; answer 7,393/245; 15,227/332	measured, thin	5–15 calls each; list prices
verifier $P(\text{fail} \mid \text{empty step})$	0.958	measured	23 of 24
verifier $P(\text{fail} \mid \text{rows step})$	0.581	measured	72 of 124; the same whether the step was good or bad
verifier structural fail on error / invalid steps	1.0 at \$0	measured	12 of 12
answer competence per (agent, category)	logit of MAS per-category accuracy + single-agent (category, tier) offset	measured, shrunk	130 MAS + 150 single-agent episodes; pseudo-count 2.0
max role calls / replans / retries	8 / 1 / 1	measured (configuration)	the real orchestrator's defaults
$P(\text{good} \mid \text{rows})$ per data agent	0.60 / 0.75 / 0.85	assumed	not identifiable from the logs; ordered like the tier accuracies
coverage $\rightarrow$ answer logit	$1.5 \times (\text{coverage} - 0.571)$	assumed (slope); centre self-calibrated	centre = `never_verify`'s expected coverage
task-level logit noise	sd 0.4	assumed	
worker "respond" on the prefix call	disallowed	assumed	first worker call is always data-2 and always queries
verify-2 (stage 2)	$P(\text{flag} \mid \text{correct})$ 0.30, same recall	assumed	
data-4 (stage 2)	rows 0.85 / good 0.80, cheap price	assumed	
answer-4 (stage 2)	mid-grade on 8 categories, cheap price	assumed	
schema_lookup (stage 3)	next worker step cannot error	assumed	priced as a verifier-sized cheap call
aggregate_calc (stage 3)	$+0.34$ coverage on aggregation categories	assumed	same price
soql_aggregate, date_window (stage 4)	$+0.15$ $P(\text{good} \mid \text{rows})$ on their categories	assumed	same price
decompose_fine (stage 4)	re-plan into 3–4 finer sub-queries	assumed	same price
planner v2 plan length $P(1..4)$	(0.0, 0.1, 0.4, 0.5)	assumed	stage 4

The run configuration is in Table 24.

R Full per-day tables

Table 28: The continual (cgrpo) arm day by day, generated from `days.jsonl` and `gates.jsonl`. * marks an event day. Served quality is the simulator's expected quality; $\bar{\lambda}$ is the served policy's learned price; ESS is the per-decision effective sample size over logged decisions; "truth" is whether the candidate was acceptable on the simulator.

day	stage	served q	cost	J	$\bar{\lambda}$	gate	DR quality [90% CI]	DR cost hi	ESS	truth
1	g0-baseline	0.405	0.0821	0.302	0.00	REVERT	0.311 [0.198, 0.489]	0.0410	61/199	ok
2	g0-baseline	0.375	0.0776	0.277	0.00	REVERT	0.583 [0.353, 0.832]	0.0790	130/394	BAD
3	g0-baseline	0.361	0.0834	0.256	0.00	REVERT	0.484 [0.358, 0.621]	0.0763	199/606	BAD
4	g0-baseline	0.403	0.0797	0.303	0.00	REVERT	0.401 [0.321, 0.495]	0.0483	298/805	ok
5	g0-baseline	0.408	0.0824	0.305	0.00	REVERT	0.389 [0.309, 0.474]	0.0494	356/991	ok
6	g0-baseline	0.334	0.0813	0.233	0.00	REVERT	0.308 [0.213, 0.415]	0.0572	302/999	ok
7	g0-baseline	0.402	0.0822	0.299	0.00	REVERT	0.292 [0.186, 0.411]	0.0547	292/1013	ok
8	g0-baseline	0.343	0.0814	0.241	0.00	REVERT	0.312 [0.228, 0.415]	0.0703	259/1002	ok
9*	g1-reprice	0.439	0.0802	0.371	0.00	REVERT	0.406 [0.094, 0.784]	0.1109	42/192	ok
10	g1-reprice	0.375	0.0718	0.315	0.00	REVERT	0.594 [0.377, 0.802]	0.0863	81/386	ok
11	g1-reprice	0.379	0.0805	0.311	0.00	REVERT	0.483 [0.259, 0.690]	0.0741	116/587	ok
12	g1-reprice	0.385	0.0755	0.321	0.00	REVERT	0.488 [0.342, 0.637]	0.0763	158/788	ok
13	g1-reprice	0.381	0.0772	0.316	0.00	PROMOTE	0.552 [0.415, 0.694]	0.0790	190/995	ok
14	g1-reprice	0.482	0.0898	0.407	0.81	REVERT	0.553 [0.433, 0.687]	0.0759	254/1070	ok
15	g1-reprice	0.491	0.0874	0.417	0.81	REVERT	0.536 [0.436, 0.629]	0.0761	347/1143	ok
16	g1-reprice	0.479	0.0864	0.406	0.81	REVERT	0.550 [0.459, 0.645]	0.0757	473/1200	ok
17*	g2-agents	0.472	0.0645	0.419	0.81	REVERT	0.528 [0.372, 0.658]	0.0744	143/283	ok
18	g2-agents	0.420	0.0756	0.359	0.81	REVERT	0.484 [0.378, 0.597]	0.0812	199/561	BAD
19	g2-agents	0.449	0.0700	0.392	0.81	REVERT	0.422 [0.344, 0.525]	0.0763	236/829	BAD
20	g2-agents	0.458	0.0699	0.401	0.81	REVERT	0.443 [0.364, 0.518]	0.0788	307/1101	BAD
21	g2-agents	0.470	0.0768	0.408	0.81	PROMOTE	0.442 [0.370, 0.518]	0.0747	408/1395	ok
22	g2-agents	0.531	0.0870	0.461	0.75	REVERT	0.445 [0.380, 0.504]	0.0805	509/1461	BAD
23	g2-agents	0.531	0.0857	0.461	0.75	PROMOTE	0.501 [0.443, 0.556]	0.0751	570/1507	ok
24	g2-agents	0.552	0.0869	0.482	1.03	REVERT	0.537 [0.482, 0.579]	0.0735	900/1593	ok
25*	g3-tools	0.519	0.0883	0.461	1.03	REVERT	0.561 [0.483, 0.641]	0.0830	339/345	ok
26	g3-tools	0.524	0.0875	0.466	1.03	REVERT	0.558 [0.503, 0.638]	0.0775	665/680	ok
27	g3-tools	0.559	0.0914	0.499	1.03	PROMOTE	0.568 [0.515, 0.615]	0.0753	997/1022	ok
28	g3-tools	0.511	0.0788	0.459	0.77	PROMOTE	0.541 [0.499, 0.590]	0.0778	1275/1371	ok
29	g3-tools	0.548	0.0971	0.484	0.66	REVERT	0.560 [0.519, 0.612]	0.0814	1410/1736	ok
30	g3-tools	0.535	0.0910	0.475	0.66	PROMOTE	0.572 [0.516, 0.626]	0.0802	1363/1747	ok
31	g3-tools	0.543	0.0832	0.488	0.53	PROMOTE	0.576 [0.519, 0.659]	0.0801	1368/1774	ok
32	g3-tools	0.561	0.0872	0.503	0.41	REVERT	0.581 [0.527, 0.651]	0.0765	1459/1809	ok
33*	g4-skills	0.518	0.0941	0.449	0.41	REVERT	0.472 [0.394, 0.583]	0.0830	412/432	ok
34	g4-skills	0.470	0.0874	0.407	0.41	REVERT	0.452 [0.390, 0.518]	0.0811	810/848	BAD
35	g4-skills	0.585	0.1041	0.509	0.41	REVERT	0.478 [0.418, 0.537]	0.0818	1165/1277	BAD
36	g4-skills	0.542	0.0986	0.471	0.41	REVERT	0.512 [0.465, 0.574]	0.0829	1540/1720	ok
37	g4-skills	0.539	0.1012	0.465	0.41	REVERT	0.522 [0.480, 0.573]	0.0825	1959/2164	ok
38	g4-skills	0.506	0.0900	0.441	0.41	REVERT	0.570 [0.512, 0.630]	0.0842	1824/2145	ok
39	g4-skills	0.505	0.0926	0.437	0.41	PROMOTE	0.600 [0.536, 0.666]	0.0771	1765/2142	ok
40	g4-skills	0.555	0.0881	0.491	0.88	REVERT	0.603 [0.547, 0.661]	0.0816	1789/2103	BAD

Table 29: Stationary held-out J (mean over the last four days of each stage), default schedule.

arm	g0	g1	g2	g3	g4
oracle	0.345	0.460	0.445	0.458	0.466
continual (cgrpo on replay, gated)	0.283	0.427	0.429	0.453	0.448
continual (AWR+IPS, gated)	0.294	0.363	0.342	0.330	0.321
frozen (trained days 1–5)	–	0.411	0.370	0.374	0.350
static router	–	0.385	0.352	0.349	0.316
router re-learned per stage	–	0.389	0.369	0.346	0.363
`always_verify`	0.329	0.426	0.389	0.397	0.376
`never_verify`	0.368	0.466	0.423	0.434	0.428

Table 30: Candidates the gate stopped that were in fact over budget (continual cgrpo arm; the AWR arm had none).

day	stage	candidate true quality / cost	incumbent true quality / cost	over budget	gate said
2	g0	0.482 / $0.1009	0.382 / $0.0794	yes	REVERT
3	g0	0.470 / $0.0981	0.382 / $0.0794	yes	REVERT
18	g2	0.515 / $0.0974	0.467 / $0.0721	yes	REVERT
19	g2	0.508 / $0.0953	0.467 / $0.0721	yes	REVERT
20	g2	0.528 / $0.0996	0.467 / $0.0721	yes	REVERT
22	g2	0.527 / $0.0962	0.512 / $0.0926	yes	REVERT
34	g4	0.521 / $0.0966	0.513 / $0.0914	yes	REVERT
35	g4	0.522 / $0.0951	0.513 / $0.0914	yes	REVERT
40	g4	0.523 / $0.0954	0.517 / $0.0904	yes	REVERT

Table 31: SkillOrchestra-style handbooks: when fitted, on how many episodes, the chosen λ_c and the SN-IPS estimate that chose it (its ESS of 96 shows why the choice is noise).

handbook	fitted on day	episodes (learn / tune)	λ_c	SN-IPS quality / cost at λ_c (ESS)
static	5	384 / 96	0.0	1.000 / $0.0787 (4/96)
relearn g1	11	192 / 96	0.0	0.597 / $0.0911 (7/96)
relearn g2	19	192 / 96	20.0	1.000 / $0.0278 (1/96)
relearn g3	27	192 / 96	0.0	0.705 / $0.0407 (3/96)
relearn g4	35	192 / 96	0.0	0.750 / $0.0690 (8/96)

S Reproduction

All CPU pipelines are seeded end to end and need only `numpy`; the CRMArena-Pro offline script is byte-for-byte deterministic on the committed log. From the repository root:

```
pip install -r requirements.txt
bash scripts/run_v0.sh              # sim: collect -> train -> gate -> drift -> retrain -> gate
bash scripts/run_sim_seeds.sh 0 1 2 3 4   # the same pipeline over five replicate seeds
head -150 traces/crmarena_b2b_counterfactual.jsonl > /tmp/b2b_50.jsonl   # the first 50 tasks
python scripts/crmarena_offline.py --log /tmp/b2b_50.jsonl --out results/crmarena   # no API calls
python scripts/crmarena_offline.py --log traces/crmarena_b2b_counterfactual.jsonl \
    --spend traces/crmarena_b2b_counterfactual.spend.jsonl --out results/crmarena_300
python scripts/crmarena_offline.py --log traces/crmarena_b2c_counterfactual.jsonl \
    --spend traces/b2c/spend.jsonl --out results/crmarena_b2c
python -m pace.experiments.taubench.run analyze \
    --log traces/taubench_retail_counterfactual.jsonl --out results/taubench/retail_RESULTS.md
python -m pace.experiments.taubench.run analyze \
    --log traces/taubench_airline_counterfactual.jsonl --out results/taubench/airline_RESULTS.md
python -m pace.experiments.evoharness.run analyze --traces-dir traces/evoharness \
    --out results/evoharness/tools_axis_150.json
python -m pace.experiments.mas.run analyze --log results/mas/mas_b2b.jsonl \
    --single-agent-log traces/crmarena_b2b_counterfactual.jsonl --out results/mas/RESULTS.md
python -m pace.experiments.ssr.run analyze --log traces/ssr/ssr_b2b_tree.jsonl \
    --out results/ssr/RESULTS.md --csv results/ssr/frontier.csv
python -m pace.experiments.mas_continual.run_continual --out results/mas_continual
python -m pace.experiments.mas_continual.run_continual --schedule mild \
    --out results/mas_continual/mild
python -m pace.experiments.mas_continual.analyze --run results/mas_continual \
    --mild results/mas_continual/mild --out results/mas_continual/RESULTS.md
python -m pytest tests -q
```

The GPU runs (`pace/llm/`) each ran once in one A100-40GB container; their training logs, run metadata, SFT reports and evaluation tables are committed under `results/llm/`. The real-harness collections (`pace.collect --harness crmarena --mode counterfactual`, the MAS and SSR `run` commands) require the benchmark checkout, its sandbox credentials and a model API key, and run under the budget check of Section 3.5. The figures in this paper are regenerated by `python paper/make_figures.py` from the committed result files, and the paper by `make` in `paper/`.

T Data license note

CRMArena-Pro is released under CC BY-NC 4.0 for research use. It is fetched at setup and never copied into the repository. The committed B2B and B2C traces (`traces/crmarena_*_counterfactual.jsonl`) are derived research artefacts (per-episode features, actions, costs, the agent's answers and queries, and the benchmark's gold answers for every task) under the same non-commercial terms; the matching call log contains token counts and prices and no benchmark data. The multi-agent and self-refinement logs are derived artefacts of the same kind. The τ-bench tasks are MIT-licensed and fetched at setup; the EvoHarnessBench logs contain our agent's actions and the service's scores and none of the service's data.